

GENERAL INSTRUCTION

- **Authors:** Please check and confirm whether the name of the corresponding author is correct as set.
- **Authors:** When you submit your corrections, please either annotate the IEEE Proof PDF or send a list of corrections. Do not send new source files as we do not reconvert them at this production stage.
- **Authors:** Carefully check the page proofs (and coordinate with all authors); additional changes or updates **WILL NOT** be accepted after the article is published online/print in its final form. Please check author names and affiliations, funding, as well as the overall article for any errors prior to sending in your author proof corrections. Your article has been peer reviewed, accepted as final, and sent in to IEEE. No text changes have been made to the main part of the article as dictated by the editorial level of service for your publication.
- **Authors:** Per IEEE policy, one complimentary proof will be sent to only the Corresponding Author. The Corresponding Author is responsible for uploading one set of corrected proofs to the IEEE Author Gateway.

QUERIES

- Q1. Author: Supplementary material is cited on pages 11 and 15. However, the material was provided in the source files. If the material is supplemental then please provide a standalone PDF to supplemental@ieee.org and annotate the PDF accordingly or rewrite the text to clarify that the material can stay in the article.
- Q2. Author: Please confirm or add details for any funding or financial support for the research of this article.
- Q3. Author: Please check and confirm whether the author affiliations in the first footnote are correct as set.
- Q4. Author: Please provide complete bibliographic details for Refs. [25], [30], [121], [123], [130], and [135].
- Q5. Author: Please provide page range for Ref. [33], [175].
- Q6. Author: Refs. [76] and [133] were identical to [169] and [175]. We have deleted [169] and [175]. Please check and confirm whether the renumbered references are correct as set.
- Q7. Author: Please update Ref. [82], [83], [110], [111], [146], [148], [149], [170], and [173] if already published.
- Q8. Author: Please provide full page range for Ref. [153].

Surface Offsetting: A Survey From Geometric Construction to Neural Implicit Representations

Xuyi Zhao¹, Graduate Student Member, IEEE, Yuanrui Yang, Jianmin Wang², and Enya Shen³

(Survey Paper)

Abstract—Surface offsetting is a fundamental geometric operation in computer-aided design, manufacturing, robotics, and computational physics. Despite its conceptual simplicity, generating offset surfaces robustly and efficiently for complex and irregular geometries remains a persistent challenge, hindered by self-intersections, topological inconsistencies, and feature degradation. This paper reviews recent advances in the field of surface offsetting, offering a structured overview of the evolution of techniques from classical constructive methods to contemporary neural implicit representations. Our survey addresses a gap in the literature, as prior foundational reviews over the past two decades in this field focused primarily on parametric methods. We introduce a taxonomy that organizes existing algorithms into five principal classes: Constructive, Spatial Discretization, Optimization-based, Field-based, and Learning-based approaches. An analysis of 46 representative algorithms reveals trade-offs in algorithm design: achieving both geometric accuracy and topological correctness proves difficult, and representation choices introduce inherent complexity constraints. We identify open problems including offset generation for surfaces with open boundaries, preservation of thin features, and resolution of self-intersections in concave regions. These challenges point toward promising research directions, including extending classical offset theory to non-manifold and open-boundary domains, developing scalable geometric predicates, and designing hybrid neural representations that disentangle distance-field smoothness from geometric sharpness.

Index Terms—Surface offsetting, implicit surface, geometric modeling, deep learning, NURBS, signed distance field.

I. INTRODUCTION

SURFACE offsetting generates new surfaces at a specified distance from original geometry, as illustrated in Fig. 1. This operation creates dilations when applied outward and erosions when applied inward. Surface offsetting is a fundamental and important operation in geometry processing. In Computer-Aided Manufacturing (CAM) and additive manufacturing, this operation generates gouge-free Computer Numerical Control (CNC)

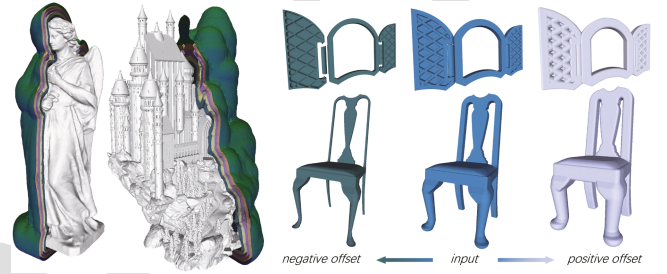


Fig. 1. Offset surface construction results on 3D models. **Left:** Multi-layer offset surfaces on sculpture and architectural models with progressive offset distances as fraction of the largest bounding box diagonal length. The offset layers are visualized as blue ($\delta = 1.0\%$), green ($\delta = 2.0\%$), purple ($\delta = 3.0\%$), pink ($\delta = 4.0\%$), and teal gradient ($\delta = 5.0\%$). **Right:** Offset construction on window and chair models with negative offset ($\delta = -1.5\%$) and positive offset ($\delta = +1.5\%$), ground truth shown in the center.

tool paths by compensating for the cutter radius [1], [2], [3]. Additionally, offsetting algorithms enable the creation of support-free hollow structures to reduce material consumption [4], [5], and define variable-thickness shells that satisfy structural constraints [6], [7]. In Computational Fluid Dynamics (CFD) and biomedical modeling, the construction of high-quality boundary layer meshes and shell structures relies on robust offset computation to maintain simulation fidelity [8], [9]. In motion planning, offset surfaces define safety margins around obstacles, establishing collision-free zones for navigation tasks [10], [11], [12].

The mathematical foundation of surface offsetting can be understood through morphological operations. An offset surface at distance δ can be interpreted as a special case of the Minkowski sum operation, where dilation and erosion are formulated as set addition and subtraction operations [13]. Geometrically, this is equivalent to sweeping a sphere along the original surface, with the sphere's center tracing the surface while its boundary defines the offset surface.

A. Related Work

The foundational literature on offset surfaces has been systematically reviewed. Pham [14] summarized methods preceding 1992, which was subsequently complemented by Maekawa [15] who reviewed work from 1992 to 1999. Furthermore, Kulczycka and Nachman [16] provided a comparison of B-spline offset

Received 24 November 2025; revised 2 March 2026; accepted 7 March 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3309000 and in part by the Ministry of Industry and Information Technology of the People's Republic of China under Grant 20241670004 and Grant WTH2025104. Recommended for acceptance by J. Hou. (Corresponding author: Enya Shen.)

Xuyi Zhao is with the College of Software, Nankai University, Tianjin 300350, China (e-mail: zhaoxuyi@mail.nankai.edu.cn).

Yuanrui Yang, Jianmin Wang, and Enya Shen are with the School of Software, Tsinghua University, Beijing 100084, China (e-mail: yyr24@mails.tsinghua.edu.cn; jimwang@tsinghua.edu.cn; sheneya@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TVCG.2026.3676903

approximation methods in 2002. These surveys primarily focused on parametric representations, including Non-Uniform Rational B-Splines (NURBS) and B-spline surfaces, and related numerical methods.

Our work expands the scope to encompass diverse geometric representations including point clouds, meshes, voxels, and implicit representation, while also incorporating the latest deep learning-based approaches to surface offsetting.

B. Scope

This survey provides a systematic examination of surface offsetting methodologies spanning four decades of development. Section II establishes the mathematical foundations by formalizing surface representations and offset definitions, while identifying the geometric singularities and topological degeneracies that fundamentally constrain algorithmic design. Section III presents a comprehensive taxonomy organizing methods into five principal classes based on their computational paradigm: Constructive approaches rooted in solid modeling, Spatial Discretization methods leveraging volumetric representations, Optimization-based formulations enforcing geometric constraints, Field-based techniques constructing implicit distance functions, and Learning-based methods employing neural implicit representations. Section IV introduces a process-oriented perspective, analyzing the four key computational stages that are common across all paradigms. Section V synthesizes these approaches through a comparative analysis across efficiency, quality, and robustness dimensions, utilizing structured tables to reveal fundamental performance trade-offs. Section VI discusses common failure cases, such as topological merging and feature erosion, that persist across all method classes. Section VII concludes the survey by identifying critical open challenges and delineates promising research directions in hybrid neural representations, scalable geometric predicates, and generalized offset theories.

II. REPRESENTATION AND OFFSETTING

This section first formalizes the various representations of surfaces and defines the surface offset operation. It then discusses the major challenges inherent in offsetting. To ensure clarity and consistency, Table I details the definitions for key terms and abbreviations referenced within this paper.

A. Surface Representation

For geometric operations such as surface offsetting, the representation decision reflects a balance between geometric exactness and computational tractability.

As illustrated in Fig. 2, these formats feature various conversion pathways. The central role of interconversion in offsetting workflows is to decouple the computational requirements of the operation from the representational demands of acquisition and output.

Point clouds consist of discrete coordinate samples, providing a direct representation of scanned data [22]. However, they lack topological or structural information, thereby requiring complex

TABLE I
KEY TERMINOLOGY AND NOTATION USED THROUGHOUT THIS SURVEY

Symbol	Full Term / Description
δ	Offset distance; signed displacement along the normal.
d_H	Hausdorff distance; maximum deviation between offset surface and ground-truth.
d_C	Chamfer distance; average bidirectional deviation between sampled points.
S	Original surface in $\mathbb{R}^3$.
$\mathbf{r}(u, v)$	Parametric surface point.
$\mathbf{n}(u, v)$	Unit normal vector at $\mathbf{r}(u, v)$.
Abbreviations	
ADF	Adaptively Sampled Distance Field; hierarchical octree-based SDF.
CSRBF	Compactly Supported RBF; locally supported RBF for implicit fitting.
DC	Dual Contouring; dual-grid isosurface extraction using Hermite samples.
GSD	Generalized Signed Distance; diffusion-based signed field.
GWN	Generalized Winding Number; implicit indicator field for solid inference.
LDI	Layered Depth Image; image-based format with multiple depth layers.
LDNI	Layered Depth-Normal Image; LDI extended with normals for offsetting.
LDF	Local Distance Field; locally computed field for precise offset control.
MC	Marching Cubes; classical voxel-based isosurface extraction.
NMAE	Normal Mean Absolute Error; mean deviation between predicted and true normals.
N-Score	Normal Stability Score; ratio of normals preserved within a θ -threshold.
PSR	Poisson Surface Reconstruction; implicit reconstruction via Poisson equation.
RBF	Radial Basis Function; implicit representation via scattered point interpolation.
SAL	Sign Agnostic Learning; neural SDF training without explicit sign supervision.
SDF	Signed Distance Field; scalar field defining an implicit surface (zero level set).
UDF	Unsigned Distance Field; magnitude-only field for open or defective geometry.

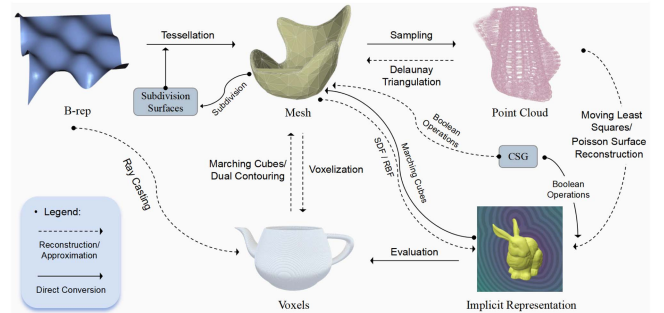


Fig. 2. Common 3D surface representations and conversion pathways. We organize formats by their mathematical structure and typical use cases. Solid arrows show direct conversions; dashed arrows show reconstruction or approximation methods. Conversion operations include fundamental processes such as tessellation and sampling, alongside methods like Delaunay Triangulation [17], Moving Least Squares [18], and Poisson Surface Reconstruction [19]. Volumetric pipelines support iso-surface extraction via methods such as Marching Cubes [20] and Dual Contouring [21].

post-processing such as surface reconstruction [23]. Meshes, as the ubiquitous representation, achieve compact storage and flexibility for arbitrary topology, but their piecewise-linear approximation fundamentally limits the geometric fidelity of smooth surfaces [24], [25]. Voxel models, based on spatial discretization, introduce quantization error at the surface boundary, often manifesting as aliasing or artifacts [26], [27]. Boundary Representation (B-Rep) encodes parametric surfaces and explicit topological relationships, making it the de facto standard for high-precision operations in CAD; its complexity lies in the demanding data structures and the difficulty of accommodating

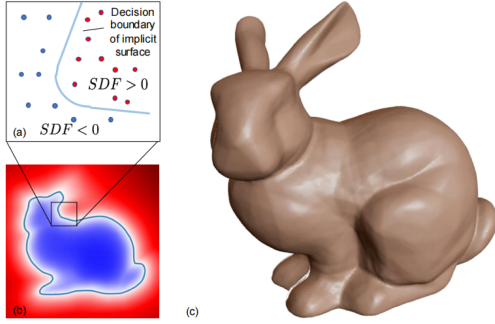


Fig. 3. Visualization of SDF. (a) The underlying implicit surface ($SDF = 0$) is learned from points sampled both inside ($SDF < 0$) and outside ($SDF > 0$) the shape boundary. (b) A 2D cross-section showing the continuous scalar field. (c) The 3D Stanford Bunny surface reconstructed from the zero-level set. Figure from Park et al. [42].

non-manifold geometries [25]. Constructive Solid Geometry (CSG) defines models compactly via regularized Boolean operations [28], which is intuitive for assembly modeling but hinders direct local geometric editing [29], [30]. Implicit representations [31], defined by scalar fields $f(\mathbf{x}) = 0$, offer continuity and compactness, which facilitate robust topology changes and natural surface generation [32]. They generally require explicit boundary extraction, such as conversion into mesh representations [20], [21] before being applicable to downstream geometric processing tasks.

B. Offset Surface Definition

1) *Normal Equidistant Offsetting*: The earliest concepts of generalized offset curves and surfaces were introduced by Bréchner [33], and subsequently expanded upon from the perspectives of differential geometry and algebra by Pottmann [34].

Parametric approaches define the offset surface through coordinate transformations applied to the original representation [35], [36], [37]. For a NURBS surface given by $\mathbf{r}(u, v) \in \mathbb{R}^3$ with unit normal $\mathbf{n}(u, v)$, the classical offset at signed distance d takes the form:

$$\mathbf{r}_d(u, v) = \mathbf{r}(u, v) + d \cdot \mathbf{n}(u, v). \quad (1)$$

Minkowski sum formulations provide an alternative explicit construction by defining the offset as the set-theoretic operation $\mathcal{M}_d = \mathcal{M} \oplus B_d(\mathbf{0})$, where $B_d(\mathbf{0})$ denotes a ball of radius d centered at the origin [10], [38], [39].

Implicit field strategies encode the offset surface as an isosurface of a scalar function. Signed distance field (SDF) constructs a volumetric representation where the zero-level set defines the original surface and level sets at varying thresholds correspond to offset surfaces [19], [40], [41]. The signed distance function $\phi(\mathbf{x})$ satisfies $\phi(\mathbf{x}) = 0$ on the surface $\mathcal{S}$ and maintains the Eikonal property:

$$\|\nabla \phi(\mathbf{x})\| = 1 \quad (2)$$

throughout the volumetric domain. The offset surface at a specific scalar value d is then defined as the isosurface $\{\mathbf{x} \in \mathbb{R}^3 : \phi(\mathbf{x}) = d\}$, as illustrated in Fig. 3. The sign of d determines

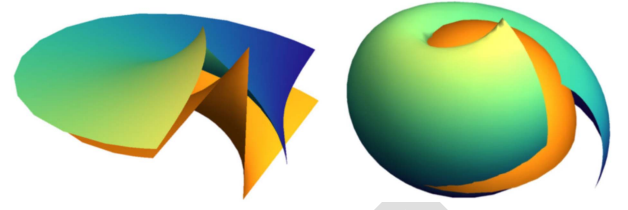


Fig. 4. Examples of generalized offset surfaces, contrasting oblique and non-equidistant cases. **Left**: An oblique offset defined by a constant distance ($\delta = 0.3$) and a constant non-normal direction ($\mathbf{e}_1 + \mathbf{e}_2 - \mathbf{n}$). **Right**: A non-equidistant offset of a half-sphere, defined by a spatially varying direction and distance $0.3 \sin(\pi(x+y))\mathbf{e}_1 + 0.3\mathbf{e}_2 - 0.3 \cos(\pi(x+y))\mathbf{n}$. Adapted from Akhrif et al. [43].

the direction of the offset: positive values ($d > 0$) correspond to outward dilation, while negative values ($d < 0$) represent inward erosion.

2) *Generalized Non-Equidistant Offsetting*: While normal equidistant offsetting remains the dominant paradigm, recent theoretical extensions have generalized the offset operation to accommodate non-uniform displacement fields. Akhrif et al. [43] introduced the concept of a generalized (non-equidistant) offset surface, which is denoted as $\mathbf{r}^0(u, v)$. In this formulation, both the offset distance and direction vary continuously over the parametric domain. Formally, with surface tangents $\mathbf{e}_1(u, v)$, $\mathbf{e}_2(u, v)$ and unit normal $\mathbf{n}(u, v) = \mathbf{e}_3(u, v)$, the generalized offset is defined as:

$$\mathbf{r}^0(u, v) = \mathbf{r}(u, v) + \sum_{i=1}^3 d_i(u, v) \mathbf{e}_i(u, v) \quad (3)$$

where $d_1(u, v)$, $d_2(u, v)$, $d_3(u, v)$ denote scalar displacement functions controlling tangential and normal components. This formulation generalizes normal offsets ($d_1 = d_2 = 0$, constant d_3) to encompass anisotropic or geodesic displacements. To approximate such non-equidistant offsets, they construct a bicubic-spline surface that interpolates sampled offset points while minimizing a third-order Sobolev semi-norm, ensuring smoothness and convergence. As illustrated in Fig. 4, the method reconstructs both an oblique generalized offset (left, constant non-normal direction) and a fully non-equidistant offset surface (right, spatially varying direction and distance), verifying the accuracy and stability of the generalized offset framework.

C. Offsetting Challenge

Surface offsetting formulation belies the operation's inherent complexity. One of the difficulties stems from the potential non-uniqueness of offset geometries, particularly at locations where the original surface exhibits tangential discontinuities. At such points, multiple normal directions may exist, creating ambiguity in determining the proper offset direction and leading to ill-defined geometric configurations.

Furthermore, even for topologically simple surfaces, the offsetting operation can induce significant topological transformations, self-intersections, and singularities. Consider the case of a multi-component assembly, such as the chain of interlinked rings shown in Fig. 5 (left). An incremental outward offset

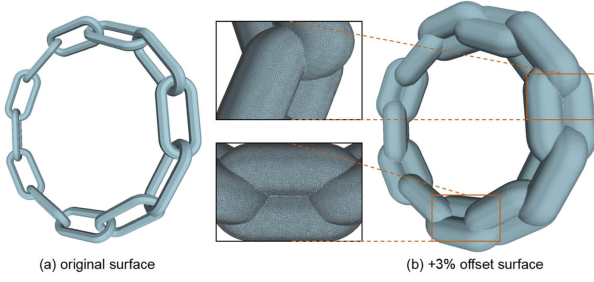


Fig. 5. Topological alteration of a multi-component assembly induced by a positive offset operation. **Left:** The original surface, comprising discrete, interlinked components. **Right:** The resulting surface after applying an offset equivalent to 3% of the model's bounding box diagonal. The expanding boundaries inter-penetrates and merge, transforming the assembly from components into a single connected manifold.

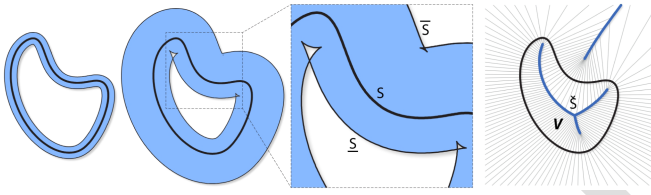


Fig. 6. Illustration of fundamental challenges in offset generation. As the offset distance increases (left), the resulting blue offset region eventually develops self-intersections (middle). The medial axis (right), depicted as thin lines, acts as the theoretical limit for the offset before this failure occurs. Figure from Musialski et al. [44].

causes the surface of each ring to dilate. Beyond a critical offset distance these expanding boundaries inter-penetrates. This operation performs a topological merge, reducing the number of connected components and producing a single connected surface, as illustrated in Fig. 5 (right).

The challenge becomes even more pronounced when considering both inner and outer offsets simultaneously, as required in many practical applications such as hollowing for NC tool path [45], [46], 3D printing or creating shell structures with specified wall thickness. As illustrated in Fig. 6, for inner offsets, the surface progressively contracts until self-intersections occur at regions where the offset distance exceeds the local feature size. This phenomenon is particularly problematic in regions with high curvature or narrow passages, where the medial axis, which represents the locus of centers of maximal inscribed spheres, provides a natural bound for feasible offset distances [44], [47].

III. METHODS CLASSIFICATION

Based on computational paradigms and representational choices, we classify surface offsetting algorithms into five principal categories, which are detailed in the following subsections.

A. Constructive Methods

Constructive methods originate from solid modeling and computational geometry, where offset surfaces are derived through

explicit geometric operations such as Minkowski sums or analytic formulations. These approaches provide guarantees of mathematical rigor and topological correctness, and in some cases yield exact closed-form solutions.

Minkowski sums: The earliest developments focused on Minkowski sums and analytic derivations. Rossignac and Requicha [10] formalized solid offsets via Minkowski sums, offering exact analytic descriptions for quadrics but restricted to convex, closed inputs. Farouki [35], [48] extended these formulations to solids of revolution, linear sweeps, and convex polyhedra, providing explicit offset equations that laid the theoretical foundation. To improve robustness for complex polyhedra, Varadhan and Manocha [11], [49] developed a topology-preserving approximation of Minkowski sums using convex decomposition and adaptive sampling, which provided Hausdorff error bounds though at quadratic complexity. Hachenberger [38] achieved the first exact Minkowski sums of non-convex polyhedra via convex decomposition and Nef polyhedra, later integrated into the CGAL library [50] as a reliable engineering tool, though with high computational cost. Campen and Kobbelt [39] introduced a polygonal boundary evaluation framework for Minkowski sums that produces watertight offset meshes. In parallel, they [51] proposed robust predicates for detecting exact and self-intersections in polygonal meshes, enabling certified correctness of offset envelopes. Wang et al. [52] further contributed an exact containment check for polyhedral envelopes, enabling certified verification of offset operations in CAD kernels.

Parametric surfaces: With the adoption of NURBS as the dominant representation in CAD, constructive offsetting was extended to parametric surfaces. Qu et al. [53] proposed a practical offsetting method for STL-format models based on vertex normal displacement, enabling lightweight implementations but prone to self-intersections. Kumar et al. [54] proposed a systematic B-Rep pipeline incorporating per-surface offsetting, gap healing, and surface stitching. Sun et al. [55] addressed local self-intersections by relocating control points and inserting knots to regularize curvature. Bastl et al. [37] proved that offsets of quadratic triangular Bézier patches remain rational and derived a direct construction using convolution and Cramer's rule. Moon [56] introduced equivolumetric offsets, generating volume-preserving generalized offsets but inevitably deviating from strict equidistance. Yue and Jia [36] extended analytic approaches to general NURBS curves and surfaces, improving precision but remaining sensitive to singularities. Wang et al. [57] proposed a heuristic offsetting scheme for Catmull-Clark subdivision surfaces, enabling smooth approximations but lacking strict distance guarantees. Jin et al. [58] proposed a general offsetting pipeline for complex car-body CAD models, integrating gap detection, Boolean trimming, G^2 patch reconstruction, and surface stitching.

Differential geometry: Differential geometry constituted another line of investigation. Abdel-Baky and Mofarreh [59] extended the notion of Bertrand offsets to timelike ruled surfaces in Minkowski 3-space, employing the E. Study map and Blaschke frames to characterize their integral invariants. Almoneef and

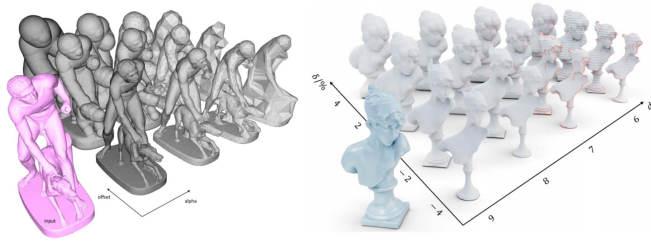


Fig. 7. Parameter sensitivity and feature preservation in offset generation. **Left:** Figure from Portaneri et al. [62] showing the α -wrapping method applied to the Hunter model where the pink mesh denotes the input geometry. The resulting offset surfaces vary with the alpha parameter (left to right: 1/600 to 1/20), which controls geometric fidelity, and with the offset distance (front to back: 1/5000 to 1/20), which controls the distance. **Right:** Figure from Wang et al. [65] comparing offset outputs under varying octree depths and offset magnitudes. The sharp feature lines are explicitly highlighted in red to verify feature preservation.

Abdel-Baky [60] investigated Bertrand offsets of slant ruled surfaces in Euclidean 3-space, deriving explicit parametrizations and invariance relations. More recently, Kaymanli et al. [61] analyzed the evolute offsets of ruled surfaces using B-Darboux frames, providing geometric characterizations though mainly theoretical.

Envelope-based: Portaneri et al. [62] introduced an α -carving shell construction built on 3D Delaunay triangulation, yielding closed manifold shells even for non-manifold or noisy inputs. As illustrated in Fig. 7 (Left), the geometric fidelity of this approach is governed by the α parameter, which balances the approximation tightness against the offset magnitude. Aubry et al. [8] realized boundary-layer meshes by recasting the problem as a discrete offset operation augmented with topological mappings, employing generalized spherical Voronoi diagrams [63] to handle complex corner extrusion. Suriyababu et al. [64] proposed voxel-based morphological shells performing dilation, erosion, and opening/closing, guaranteeing watertight outputs but not strict equidistance.

B. Spatial Discretization Methods

Spatial discretization methods approximate offsets by discretizing continuous geometry into voxel grids, layered depth images, or ray-based structures. By avoiding direct manipulation of surfaces, these methods achieve robustness against defects and suitability for parallelization, but their accuracy is constrained by resolution and memory consumption.

Ray-based: Hartquist [66] introduced a ray representation of Minkowski sums, laying the conceptual foundation for ray-based offsetting, which provided robustness for thin structures. Subsequent advances exploited GPU rasterization and structured sampling. Wang [67] formalized a parallel computing framework on rays. Martinez et al. [68] proposed chained zonotope approximations using rasterized buffer accumulation, achieving efficient offsetting of slender geometries. Shade et al. [69] introduced the concept of Layered Depth Images (LDI). Additionally, Wang [70] developed solid modeling of polyhedral objects using Layered Depth-Normal Image (LDNI) [71] on GPU. Chen and Wang [72] developed the LDNI-based offsetting, encoding

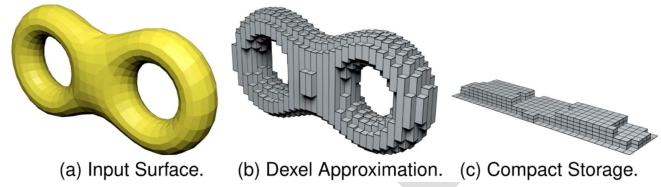


Fig. 8. Construction of the dixel data structure from explicit geometry. (a–c): The input triangle mesh (a) is sampled by intersecting it with a set of uniformly spaced parallel rays (b). These intersections are then compactly encoded as a 2D grid of events within the dixel buffer (c), a representation that facilitates efficient CSG operations. Figure from [75].

solids as layered orthogonal images that preserve both depth and normals; this approach enabled feature-aware offsetting but required dense sampling. Wang and Manocha [73] extended LDNI with structured point sampling on GPUs, accelerating offset operations, though still bounded by resolution. Chiu and Tan [74] leveraged dixel structures [1] for morphological hollowing, employing one-dimensional Boolean operations on ray segments to precisely define internal offset boundaries and support structures. Chen et al. [75] further developed an offsetting method based on dixel, which uses half-space power diagrams (see Fig. 8) though accuracy degraded near high-curvature features. Additionally, decomposition strategies can optimize the dilation process by splitting the operation into orthogonal stages.

Voxel-based: Voxel-based approaches formed another important branch. Breen and Mauch [76] presented one of the earliest volumetric offsetting frameworks, where CSG models were converted into distance and closest-point volumes, followed by iso-surface extraction to realize shaded offset surfaces. This method unified geometry and appearance handling but incurred substantial aliasing and storage cost. Li and McMains [77] demonstrated GPU-accelerated voxelized Minkowski sums, enabling robust morphological offsets of polyhedral models. Chen et al. [78] proposed a hybrid approach that explicitly samples offset points for vertices, edges, and faces, while leveraging a uniform grid structure to efficiently filter out invalid internal samples. Liu and Wang [79] proposed an intersection-free contouring method for volumetric offsets, enhancing dual contouring with geometric constraints to eliminate self-intersections, achieving scalability to large freeform meshes though at the cost of preprocessing. Chen et al. [80] introduced a two-stage framework for robust low-poly meshing, with the first stage dedicated to an offset iso-surface extraction algorithm. This stage generates meshes that are watertight, manifold, and suitable as proxies for subsequent simplification and feature alignment. Zint et al. [81] introduced topology-adaptive octree offsets to improve accuracy in curved regions, demonstrating high robustness to defective inputs such as triangle soups (see Fig. 9).

C. Optimization-Based Methods

Optimization-based methods reformulate offsetting as a constrained optimization or energy minimization problem, which allows smoothness, feature preservation, and topology control to be jointly enforced in a unified framework. This paradigm

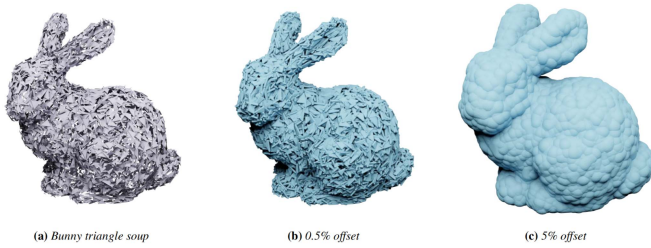


Fig. 9. Demonstration of the robustness of the topology-adaptive octree offset method by Zint et al. [81]. From left to right: the input bunny triangle soup, which is non-manifold; the resulting 0.5% offset; and the 5% offset. The method guarantees a watertight and 2-manifold output mesh even from highly defective inputs.

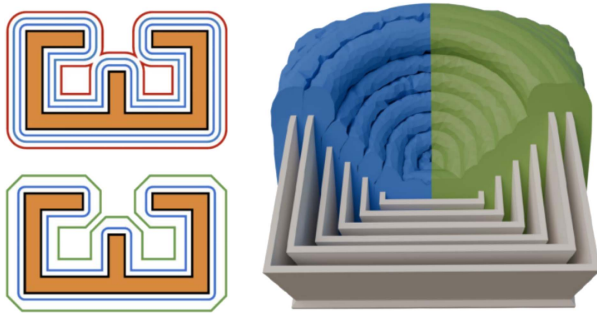


Fig. 10. Robustness comparison of the topological offset, adapted from Zint et al. [84]. **Left:** In 2D, the standard large finite offset shown in red creates self-intersections while the green topological offset preserves the structure similar to the blue ideal infinitesimal offset. **Right:** In 3D, the blue surface shows the topological offset result; the green surface represents the finite offset constructed by using multiple offset layers obtained from topological offsets as input.

offers flexibility but often incur substantial computational costs due to the scale of the optimization problems involved.

Constrained optimization: Representative works include Wang et al.'s PCO method [65], which leverages local distance fields and tetrahedral cutting with clique-based simplification to preserve sharp features. Fig. 7 (Right) demonstrates this capability, visualizing how sharp feature lines are explicitly preserved across varying octree depths and offset distances. Cao et al. introduced parallel feature-preserving framework [82] and robust offset meshing [83], combining dynamic programming with quadratic programming to enable parallel variable-distance offsets while maintaining sharp features.

Meng et al. [85] presented a particle-system-based method for computing feature-aligned polygonal offset surfaces, formulating offsetting as a constrained energy minimization that drives particles to satisfy distance and alignment constraints. Akhrif et al. [43] developed a variational bicubic spline approximation under Sobolev norms, achieving C^2 -continuous generalized offsets. Kouibia and Pasadas [86] extended spline-based offset approximation to the case of surfaces with holes, formulating the problem as a biquadratic spline reconstruction. This framework established the existence and uniqueness of solutions and validated convergence through numerical experiments.

Shell-based: Shell-based mappings also exemplify optimization driven formulations, building upon foundational work in

shell maps [87] and nested cages [88]. Jiang et al. [89] proposed bijective projection shells, ensuring one-to-one mappings between source and offset surfaces. Liu et al. [90] extended this idea with high-order shells using Bézier triangles, enhancing smoothness and uniformity in regions of high curvature. Zint et al. [84] presented Topological Offsets, embedding surfaces volumetrically and applying topological operators to guarantee homeomorphism, even under near-collision scenarios (see Fig. 10). While robust and general, optimization-based methods remain computationally demanding compared to direct constructions.

D. Field-Based Methods

Field-based methods approach offsetting by constructing scalar fields whose isosurfaces define offset geometry. By shifting the problem into a volumetric domain, these approaches achieve robustness against noise, non-manifold inputs, and severe defects, at the expense of efficiency and memory.

Distance field: Classical implicit surface reconstruction from unorganized points [23] established foundational principles for field-based offsetting. Constructing distance field [91] is a prerequisite for field-based offsetting, since offset surfaces correspond directly to isosurfaces at varying thresholds. Early efforts by Frisken et al. [92] introduced Adaptively Sampled Distance Fields (ADF), storing signed distances in an octree-like adaptive grid that concentrates resolution in regions of high curvature, thus enabling scalable offset evaluation. Multi-level partition of unity implicits [93] and feature-preserving point set surfaces [94] demonstrated that local reconstruction strategies can preserve geometric detail while maintaining global consistency. Xu and Barbič [40] proposed a practical pipeline for building SDF from polygon-soup meshes, combining voxelization and closest-point queries to produce watertight signed fields even for defective geometry.

Radial basis function: Radial Basis Function (RBF)-based methods provide a line of research. Carr et al. use RBFs to fit point sets with normals, generating signed implicit fields [95]. Liu and Wang proposed the Duplex Fitting approach [96], which employs compactly supported RBFs (CSRBFs) to fit both the zero-level and offset surfaces within a single implicit function, though large offset distances may lead to local self-intersections due to normal extrapolation. Yoo further combined voxel distance fields with RBF interpolation to build a general framework for offsetting triangular meshes [97], allowing for variable offset control via defined points as shown in Fig. 11, though with limited feature preservation in high-curvature regions and efficiency tightly coupled to voxel resolution.

Partial differential equation: Another class of methods relies on partial differential equation (PDE) solutions to construct scalar fields. Kazhdan et al. [19], [98] introduced Poisson surface reconstruction (PSR), where solving a Poisson equation yields an implicit indicator function naturally supporting offset isosurfaces. Crane et al. [99] developed the heat method for geodesic computation, later extended by Feng and Crane [41] to compute

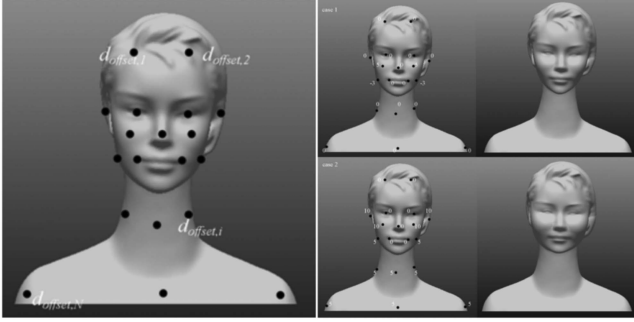


Fig. 11. Non-equidistant offsetting of a mannequin model. **Left:** Definition of control points and the specific offset distance value assigned at each vertex. **Right:** The resulting non-equidistant offset surface generated by interpolating these constraints. Adapted from Yoo [97].

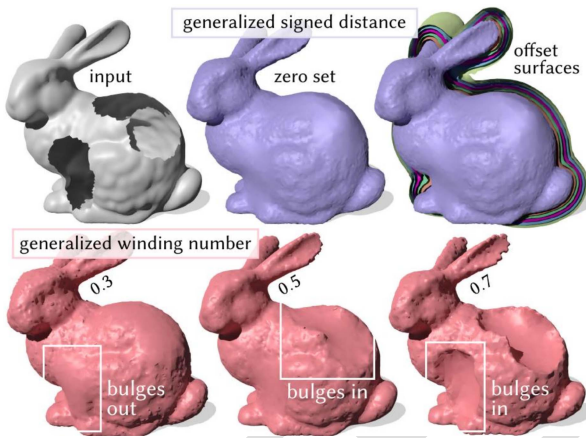


Fig. 12. A comparison of PDE-based reconstruction from a defective input, from Feng and Crane [41]. *Top row:* The SHM method uses normal information to plausibly reconstruct the surface, shown as a clean zero set and smooth offset surfaces. *Bottom row:* The Generalized Winding Number (GWN) method creates undesirable harmonic patches, resulting in topological artifacts such as bulges where the input geometry is missing.

generalized signed distances (GSD) by diffusing normal fields and solving Poisson equations, which generate offsets without prior geometric repair (see Fig. 12). These PDE-inspired strategies provide smooth, stable, and topologically consistent offsets even for highly defective geometry.

E. Learning-Based Methods

Learning-based methods, particularly neural implicit representations, have become a practical paradigm for offsetting. In this framework, neural networks approximate a continuous SDF, where isosurfaces at thresholds $f(x) = d$ correspond to offset surfaces. Unlike purely discrete approaches, neural implicit fields provide a compact and continuously sampled representation.

Neural fields: Coordinate-conditioned neural fields represent shapes as continuous functions that can be sampled at arbitrary locations [101], making isosurface extraction straightforward within the same representation. Early works such as

DeepSDF [42] and SAL [102] demonstrated that neural networks can successfully learn SDF from data. Meta-learning for neural SDFs [103] explores fast adaptation to new shapes from few observations, which is useful when evaluating multiple offset thresholds across diverse geometries under limited supervision. The effectiveness of positional encoding and Fourier features for learning high-frequency functions has been further analyzed [104], contributing to improved fidelity of neural implicit surfaces. Sitzmann et al. [105] introduced SIREN, whose sinusoidal activations allow the learned SDF to capture high-frequency geometric details, thus improving the fidelity of fine-scale offsets. Gropp et al. [106] proposed IGR, which imposes Eikonal regularization to enforce true distance function properties, yielding offset surfaces with more stable geometry and reduced artifacts. Coiffier et Béthune [100] introduced 1-Lipschitz neural distance fields, enforcing Lipschitz continuity during training to guarantee stable signed distance representation (see Fig. 13). This constraint improves the geometric fidelity and robustness of offset isosurfaces, particularly for high-curvature regions.

While SDFs characterize watertight shapes via signed distances, Unsigned Distance Fields (UDFs) extend this representation to general geometries by modeling the distance magnitude directly [107]. However, learning UDFs often encounters the challenge of vanishing gradients near the zero-level set. To mitigate this, approaches [108], [109] integrate geometric priors, such as normal alignment, geometry-guided distance representations and adaptive Eikonal constraints, to capture high-fidelity details. Strategies utilizing local shape functions have been proposed to generalize across shapes without shape-specific retraining [110]. Furthermore, zero level-set extraction methods based on double covering approaches have been introduced to resolve topological ambiguities and extract valid surfaces from these fields [111].

Geometric priors: Several hybrid approaches integrate classical geometric priors to improve offset fidelity. Prasad et al. [112] developed NURBS-Diff, a differentiable NURBS layer that enables CAD-based surfaces to be offset through gradient-based learning. Marschner et al. [113] combined CSG operations with neural SDF, ensuring that offset surfaces respect sharp Boolean features. Guo et al. [114] proposed NH-Rep, which leverages B-Rep Boolean trees within neural half-space representations. Xu et al. [115] proposed an implicit filtering framework for training neural signed distance functions directly from raw point clouds, stabilizing optimization by filtering out noisy or inconsistent samples. The method improves generalization to non-zero level sets, enabling more accurate offset surfaces under noisy or incomplete inputs. Genova et al. [116] introduced local deep implicit functions. Chabra et al. [117] developed deep local shapes for detailed reconstruction. Lin et al. [118] introduced Patch-Grid, which augments neural SDF with localized feature volumes and shared octrees, enabling robust offset generation even for thin shells and open boundaries.

Implementing the diverse methodologies detailed in this survey, leverages a diverse set of geometry processing libraries. Table II lists representative libraries adopted in surface offsetting, grouped by their primary functionality.

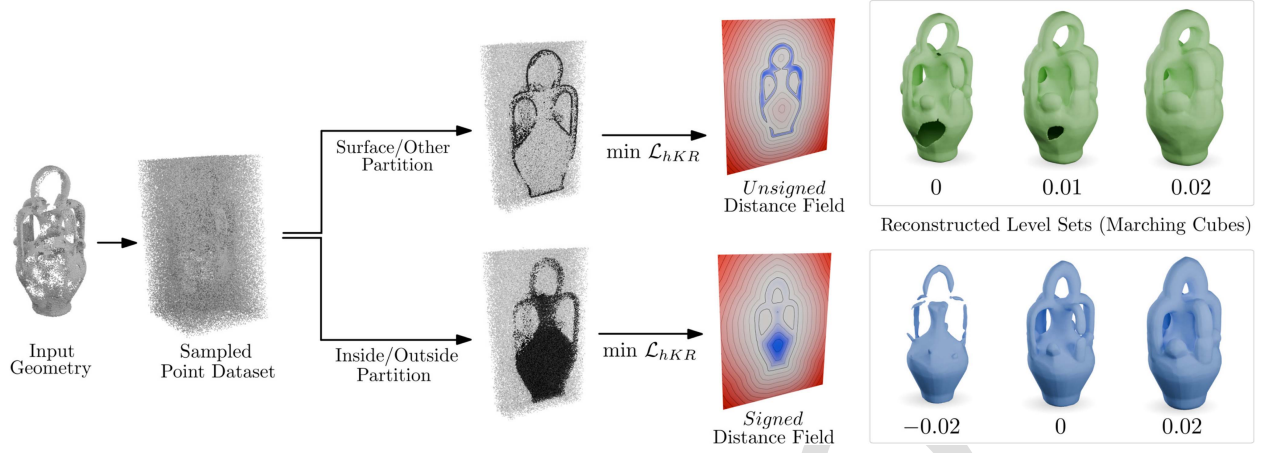


Fig. 13. The 1-Lipschitz neural distance field pipeline by Coiffier et Béthune [100], which learns distance fields by partitioning a corrupted point dataset. *Top row*: An Unsigned Distance Field is learned by partitioning points into those on the surface and those in the surrounding space, resulting in the green offset surfaces. *Bottom row*: A Signed Distance Field is learned by partitioning points as inside or outside the shape, resulting in the blue inner and outer offsets.

TABLE II
REPRESENTATIVE SOFTWARE LIBRARIES AND FRAMEWORKS UTILIZED IN
SURFACE OFFSETTING RESEARCH, CATEGORIZED BY FUNCTIONALITY

Library	Primary Purpose in Offsetting
<i>Core Geometry Processing</i>	
CGAL [119]	Robust computational geometry, AABB trees, mesh processing.
libigl [120]	Geometry processing; mesh ops (normals, winding number). Advanced mesh data structures; normals/gradients/curvature.
Geometry-Central [121]	Geometric algorithms, mesh generation, fast nearest-neighbor search.
Geogram [122]	Geometric algorithms, mesh generation, fast nearest-neighbor search.
DGtal [123]	Discrete geometry processing; distance-field construction.
pmp-library [124]	Lightweight C++ polygon-mesh processing.
<i>Meshing and Tessellation</i>	
TetGen [125]	Tetrahedral mesh generation/handling for volumetric representations.
TetWild [126]	Robust tet mesh generation from triangle soups / defective inputs.
Robust CDT [127]	Robust constrained Delaunay triangulation.
RVD (Computation) [128]	Restricted Voronoi Diagram for internal offset partitioning.
scikit-image [129]	Isosurface extraction; image-based preprocessing.
<i>Numerical Solvers and Parallelism</i>	
Eigen [130]	Linear algebra; numerical solvers; optimization.
Intel TBB [131]	Task-based parallel computation.
OSQP [132]	Quadratic programming solver (optimization-based offsets).
FMM [133]	Fast distance-field computation.
<i>Proximity Queries</i>	
ANN [134]	Approximate nearest-neighbor search; distance queries.
PQP [135]	Proximity / collision queries.
nanoflann [136]	Header-only KD-tree search in C++.
SciPy [137]	Scientific computing (optimization, spatial search) in Python.
<i>Validation</i>	
PCK [138]	Exact geometric predicates for robust ops.
Metro [139]	Error measurement for geometric accuracy.
CREATETM-MG Capstone [140]	Industrial CAD/CAE tools for validation and metrology.

IV. COMPUTATIONAL STAGES

This section provides a process-oriented perspective complementary to the method-centric taxonomy presented in Section III, revealing how algorithmic paradigms differ primarily in their stage-specific strategies and computational resource allocation.

The pipeline comprises four sequential stages with feedback loops. Table IV documents strategy choices across the five method classes, while Table V illustrates relative computational investment patterns.

A. Construction

Offset construction strategies differ in their computational dependency structures, determining how geometric information propagates from input to output and what constraints govern this transformation process.

Local independent computation: Independent local computation achieves offset generation through spatially decoupled operations where every surface point, volume cell, or field query operates on locally available information. Direct coordinate transformation methods [36], [53] compute displacements through pure functions of local geometry such as surface normals and parametric derivatives. Ray-based spatial discretization strategies [68], [73], [75] partition space into independent sampling rays or dextel structures where each ray determines occupancy through local intersection tests.

Global coupled constraints: Coupled constraint satisfaction methods [41], [82], [85], [89], [90] formulate offsetting as a globally-coupled system where constraints must be satisfied simultaneously across the entire domain. Optimization-based approaches [82], [85] couple neighboring elements through energy minimization, where particle systems or dynamic programming jointly optimize vertex positions under distance constraints. Volumetric parameterization methods [89], [90] enforce global injectivity through bijective shell constructions or tetrahedral embeddings that couple geometric validity across the entire domain. Field-based PDE solvers [41] exhibit intrinsic

To validate and benchmark these implementations, a wide array of 3D geometric datasets are employed. Table III provides a summary of the datasets commonly utilized in the surface offsetting literature.

TABLE III
DATASETS FOR SURFACE OFFSETTING AND GEOMETRIC MODELING EVALUATION

Name	Institution	Year	Scale	Format	Description
ABC Dataset [141]	NYU, TU Berlin	2019	1M+ shapes	STEP, STL, OBJ	Parametric surface reconstruction.
Fusion 360 Gallery [142]	Autodesk	2021	8,625 designs	SMT, STEP, JSON	Real CAD design sequences.
Fusion 360 Gallery Seg. [143]	Autodesk	2021	35,680 models	SMT, STEP, OBJ	Face-level segmentation for B-rep models.
FloorPlanCAD [144]	HKUST, SFU	2021	15K floorplans	SVG, PNG	Annotated CAD drawing parsing for floor plan understanding.
ModelNet [145]	Princeton	2015	12,311 shapes	OFF	40-class shape classification benchmark.
ShapeNet [146]	Stanford	2015	51,300 shapes	OFF	Large-scale annotated 3D shape repository.
PartNet [147]	Stanford	2019	26,671 shapes	OFF, JSON	Hierarchical part-level segmentation dataset.
Thingi10K [148]	NYU	2016	10K shapes	STL, OBJ	Real-world 3D-printable models.
SketchGraphs [149]	MIT, Princeton	2020	15M+ sketches	JSON	Large-scale parametric CAD sketch dataset.
DeepCAD [150]	CUHK, UCSD, MIT	2021	178,238 sequences	JSON	Parametric modeling sequence corpus.
3D-FUTURE [151]	Alibaba DAMO	2020	16,563 models	OBJ, MTL, JSON	High-quality furniture model dataset.
Princeton SB [152]	Princeton	2004	6,670 shapes	OFF	Early shape classification benchmark.
MFCAD [153]	Queen's Univ. Belfast	2020	15,468 shapes	STEP	Planar machining feature dataset.
MFCAD++ [154]	Queen's Univ. Belfast	2022	59,655 shapes	STEP	Enhanced hierarchical machining features.

TABLE IV
STAGE-SPECIFIC STRATEGIES ACROSS METHOD CLASSES

Method Class	Stage I: Construction	Stage II: Verification	Stage III: Adaptation	Stage IV: Extraction
Constructive	Parametric algebra [35], [36], [54]; NURBS control point displacement; exact local distance	Boolean trimming [54], [58]; explicit intersection detection; lightweight post-process	Per-feature offsetting [55], [58]; control point adjustment; knot insertion	Direct output; no extraction needed; already explicit
Spatial Discretization	Voxel rasterization [64], [77]; layered depth [72], [73]; octree [81], [155]; GPU-friendly	Morphological operations regularize occupancy [64]; explicit collision tests uncommon	Adaptive resolution [81]; multi-orientation sampling [73]; octree refinement	MC/DC [20], [21]; resolution-dependent quality
Optimization-based	Mixed mesh-field [65]; tetrahedral [89], [90]; variational formulation [82], [85]	Barrier functions [90]; integrated constraints [82]; collision detection and repair	Energy minimization [82], [85]; feature constraints [65]; bijective mapping [89], [90]; topological operators [84]	Projection [89]; optional remeshing [84]
Field-based	SDF construction [19], [40], [41]; RBF fitting [95], [96]; volumetric storage	Gradient monitoring [44]; implicit regularization; circumvent explicit detection	Eikonal projection; anisotropic diffusion; PDE smoothing [19], [41]; virtual closure for open boundaries	MC/DC [20], [21]; isosurface extraction; balanced construction-extraction cost
Learning-based	Neural SDF training [42], [100], [102], [105], [106]; compact parameters; hybrid representations [114], [118]	Architectural constraints; loss regularization; implicit avoidance through learned priors	Lipschitz constraints [100]; multi-resolution [114]; hybrid features [118]; Eikonal regularization [106]	Differentiable extraction [156]–[161]; MC/DC [20], [21]; fast inference; gradient-based optimization

TABLE V
COMPUTATIONAL EMPHASIS DISTRIBUTION ACROSS PIPELINE STAGES
(SYMBOLIC SCALE)

Method Class	Construction	Verification	Adaptation	Extraction
Constructive	+++	++	++	/
Spatial Discret.	+	+	+	+++
Optimization	++	++	+++	+
Field-based	++	+	++	++
Learning-based	+++	+	++	+

coupling through differential operators where heat diffusion couples all spatial locations via the Laplacian. Learning-based methods [100], [105], [114] optimize over function spaces where loss terms computed at scattered sample locations back-propagate through network parameters, creating computational dependencies spanning the entire learned representation. These coupled formulations necessitate iterative solving through gradient descent, Newton methods, or fixed-point iteration until convergence criteria indicate constraint satisfaction.

Hierarchical progressive: Hierarchical progressive construction builds offset geometry through multi-scale refinement where initial coarse approximations capture global topology before subsequent passes correct local errors. Adaptive octree

methods [81], [92] exemplify this pattern through spatially-varying subdivision where parent cell decisions determine child configurations, with refinement cascading from root to leaves based on local geometric complexity. Alpha wrapping [62] establishes conservative outer approximations through Delaunay-based carving and refinement, where initial wrapping phases provide bounds that subsequent tightening operations must respect. Subdivision-based approaches [57] encode progressive dependency through recursive refinement rules applied to control meshes, where limit surface evaluation requires hierarchical application of smoothing operators. These hierarchical strategies establish temporal dependencies through their multi-pass nature, where later stages build upon representations and decisions established in earlier phases.

B. Verification

Verification mechanisms govern how algorithms detect invalid states and trigger corrective actions during offset computation.

Verification timing: Constructive approaches [38], [39], [54] perform post-operation validation after each geometric transformation and trimming operation. Optimization-based formulations monitor constraint satisfaction at each iteration to maintain

bijection and smoothness [90]. Vertex displacements can also be solved through dynamic and quadratic programming with guaranteed collision-free solutions rather than iterative penalty adjustment [82]. Implicit methods embed continuous monitoring within their solving procedures. Distance field constructions enforce Eikonal consistency by solving a Poisson equation with a normalized diffused gradient field [41], providing intrinsic validation within the construction process rather than explicit post-checking. Neural SDF training monitors Eikonal loss violations and constrains gradient norms through regularization [106] without explicit runtime adjustment mechanisms.

Validation criteria: Validation criteria rely on detection mechanisms categorized into explicit geometric predicates and implicit field regularity. Explicit strategies for discrete representations leverage spatial acceleration structures to prune candidate pairs before resolving intersections through polynomial systems or planar projections [73], [162], [163]. These methods frequently adopt exact arithmetic based on rational number systems to prevent robustness failures [38]. Conversely, implicit formulations intrinsically maintain validity by enforcing field consistency through neural regularization limits that suppress non-physical solutions without explicit checks [105], [106], [164].

Response mechanisms: Response mechanisms address detected invalidity through preventive, corrective, or deferred strategies. Pre-computation techniques adjust input geometry via control point repositioning or constraint reformulation to ensure curvature limits are satisfied prior to generation [37], [55]. Runtime approaches dynamically repair violations during computation by iteratively projecting configurations onto feasible manifolds or adapting spatial resolution near geometric complexities [81], [89], [90], [165]. Post-processing methods alternatively tolerate intermediate errors and resolve global self-intersections through batch trimming operations after the offset construction [39], [53].

C. Adaptation

Adaptation mechanisms dynamically adjust computational strategies based on evolving solution quality and resource constraints.

Spatial refinement: Spatial refinement determines where to concentrate computational resources within the geometric domain. Error-driven strategies measure deviation between current and target geometry through distance metrics or normal alignment scores. Volumetric methods [155] compute local Hausdorff distances between offset isosurfaces and reference surfaces, subdividing octree cells where errors exceed user-specified thresholds. Feature-driven strategies identify geometric detail requiring enhanced resolution regardless of global error distributions. Sharp edge detection through dihedral angle thresholding [55], [58] guides independent offset computation along crease curves, preventing smoothing artifacts at feature boundaries. Topology-adapted octree refinement increases resolution near topologically intricate or high-curvature regions to maintain connectivity under negative offsets [81].

Temporal adaptation: Temporal and hierarchical adaptation govern how algorithms regulate computational effort over time and across resolution levels. Fixed iteration schemes execute predetermined numbers of solving or optimization steps, providing predictable runtime but risking under-refinement or wasted computation. Adaptive schemes monitor convergence indicators such as residual norms in PDE-based solvers [19] or energy decrease rates in optimization formulations [85], with stopping criteria tuned to balance runtime and accuracy. Resource allocation across spatial hierarchy follows similar principles. Uniform discretizations such as dense voxelization [77] and uniform LDNI sampling [72] distribute memory evenly, enabling straightforward parallelization but wasting capacity in empty regions. Adaptive hierarchies [81], [155] concentrate computation near regions of geometric complexity through octree subdivision, achieving logarithmic query complexity at the cost of dynamic memory management overhead. Neural implicit representations [100], [106] adopt coarse-to-fine scheduling strategies, first capturing global structure at low resolution and progressively refining surface detail through increased sampling density. Hybrid neural-field formulations [114], [118] further decouple topology capture from feature representation, using global fields for connectivity and localized patches for sharp-feature encoding.

D. Extraction

Implicit and volumetric representations require explicit mesh extraction for downstream use. This stage converts intermediate representations into valid, high-quality triangle meshes through isosurface extraction.

Classical extraction: Classical algorithms such as Marching Cubes (MC) [20] became the standard techniques for converting voxelized signed distance fields into polygonal representations. Though MC suffers from feature blurring, feature-sensitive extraction [27] and Dual Contouring (DC) [21] improve sharp feature preservation by leveraging directed distances and Hermite data respectively. While Manifold Dual Contouring [166] addresses topological ambiguities via vertex clustering, approaches like the method of Rashid et al. [167] further ensure watertightness and manifold properties by decomposing ambiguous grid cubes into tetrahedra, thereby eliminating geometric singularities inherent in cubic discretizations.

Neural extraction: Learning-based extensions integrate neural priors into classical extraction pipelines, such as Deep Marching Cubes [156], Neural Marching Cubes [168], and Neural Dual Contouring [157], enhancing detail recovery and robustness while preserving the efficiency of traditional isosurface algorithms. Beyond these variants, recent advances have emphasized adaptivity and differentiability. Deep Marching Tetrahedra (DMTet) [158] introduces a deformable tetrahedral formulation that supports topology changes and gradient-based optimization, FlexiCubes [159] augments dual-style extraction with local degrees of freedom to improve mesh quality and topological fidelity, McGrids [160] employs Monte Carlo sampling to construct adaptive tetrahedral discretizations with reduced memory

and query costs, and TetWeave [161] integrates Delaunay tetrahedralization with directional signed distance fields to guarantee watertight, manifold, and self-intersection-free outputs.

V. COMPARATIVE ANALYSIS

Literature selection: We queried major digital libraries, including IEEE Xplore, ACM Digital Library, SpringerLink and Google Scholar, spanning publications from January 1985 to November 2025. From an initial retrieval of articles, we filtered the literature to select the 46 representative algorithms analyzed in this survey. Our inclusion criteria required that the methods (1) focus primarily on the generation, approximation, or optimization of 3D offset surfaces, (2) introduce a novel computational paradigm, data structure, or mathematical formulation rather than simply applying existing tools to downstream tasks, and (3) serve as representative methods, spanning from historical mathematical foundations to the current state-of-the-art within their respective categories. To address incremental research, we merged near-duplicate works from the same authors, retaining only the most technologically mature or comprehensive version (e.g., a journal extension over a preliminary conference paper). Furthermore, for hybrid methodologies that cross our five principal classes, classification was determined by the algorithm’s dominant computational mechanism driving the offset generation. A detailed description of our search strategy, inclusion/exclusion criteria, and classification guidelines is provided in the Supplementary Material.

Evaluation framework: To provide a structured comparison of these methodologies, we evaluate them along three critical dimensions: computational efficiency, output quality, and robustness. These metrics reveal fundamental balance that define the practical utility of different methodological approaches. Due to differences in reporting standards, hardware environments, and experimental setups in the existing literature, the comparative metrics presented in Tables VI, VII, and VIII utilize a tiered, symbolic grading system (detailed definitions provided in the Supplementary Material). Any missing metrics or architectural specifics are denoted with “-”. These tables should be interpreted as a qualitative synthesis of capabilities.

This classification is predicated on the rationale and assumptions as follows:

- For Table VI, we classify methods by their adopted spatial acceleration structures (e.g., tree-based or adaptive grids, tiers A1–A5), iteration levels (tiers I1–I3), and device architecture dependencies (C, G, and H). Evaluating fundamental architectural traits isolates the intrinsic algorithmic efficiency from hardware-specific execution time discrepancies.
- For Table VII, the classification is based on the principle that the precision of foundational geometric operators, namely normal estimation (tiers N0–N5), closest-point queries (tiers Q1–Q5), and the spatial discretization levels (tiers D1–D3) constrain the achievable offset surface fidelity. The accuracy classes (tiers A1–A3) are derived from the reported distance metrics. These values were

normalized against the bounding box scale of the input model in the original literature to ensure scale invariance. This represents the standard evaluation practice across the majority of the reviewed studies. Exceptions utilizing alternative normalization scales are explicitly identified with superscripts and detailed in the table notes. Furthermore, the assessment of topological validity evaluates the avoidance of self-intersections, the preservation of manifoldness, and the generation of watertight boundaries, which influence the output’s viability for subsequent tasks.

- For Table VIII, we establish this evaluation across three dimensions. By analyzing a method’s response to defective inputs (non-manifold, non-watertight, and degenerate geometries), we evaluate its dependence on strict topological correctness. Furthermore, the examination of Extreme Parameters assesses algorithmic stability under boundary conditions: infinitesimal offsets ($\delta \rightarrow 0$) evaluates a method’s numerical stability and spatial resolution limits when handling extremely small displacements; negative offsets ($\delta < 0$) assesses whether the method supports inward erosion; and normal flips (Flip-N) tests the algorithm’s reliance on local derivatives. Post-Processing dependency clarifies whether a method autonomously outputs valid geometry or relies on additional remeshing.

A. Efficiency

Table VI categorizes the performance factors, including representation conversion pipelines, acceleration strategies and prescribed iteration levels.

Solver mechanism: Procedural and constructive methods [10], [54], [62] rely on explicit geometric operation. These approaches benefit from deterministic execution paths that typically run on CPUs. In contrast algorithms formulated as linear systems [41] depend on sparse matrix solver efficiency where performance is bound by the algebraic structure and equation solving speed. Non-linear optimization frameworks [82], [85] demand higher frequency iterative updates dependent on optimization strategies necessitating GPU or hybrid architectures.

Representation conversion: Approaches employing intermediate volumetric or implicit representations [11], [40], [80], [106], [113], [114], [155] necessitate a multi-stage conversion chain involving voxelization field calculation and isosurface extraction. Although these indirect pipelines provide superior mechanisms for maintaining global topological validity the additional discretization and reconstruction steps introduce computational overhead that scales with the resolution of the intermediate grid.

Spatial acceleration strategies: CPU-dominant algorithms frequently employ hierarchical structures, including AABB or KD-trees [80], [85], which facilitate fast nearest neighbor queries and collision detection despite moderate construction costs. However, traversing irregular trees induces incoherent memory access patterns, which degrade the efficacy of massive parallelism. Consequently, GPU-accelerated methods typically prioritize data regularity and memory coalescing by leveraging

TABLE VI
REPRESENTATION CONVERSION PIPELINES AND COMPUTATIONAL PERFORMANCE FACTORS

Ref.	Representation and Conversion				Acceleration	Iteration	Device
	Input	Offset Format	Output	Conversion			
[155]	Mesh	Voxel	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Mesh	A1	11	C
[82]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A1	12	C
[79]	Mesh	Implicit (SDF)	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Implicit (SDF) $\rightarrow$ Mesh	A1	–	C
[64]	Mesh / Point Cloud	Voxel	Mesh	Mesh / Point Cloud $\rightarrow$ Voxel $\rightarrow$ Mesh	A1	–	C
[80]	Mesh	Implicit (UDF)	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Implicit (UDF) $\rightarrow$ Mesh	A2	12	C
[85]	Mesh / Point Cloud / Implicit	Point Cloud	Mesh	Mesh / Point Cloud / Implicit $\rightarrow$ Point Cloud $\rightarrow$ Mesh	A2	12/3	C
[89]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A2	–	C
[62]	Mesh / Point Cloud	Mesh	Mesh	Mesh / Point Cloud $\rightarrow$ Mesh	A2	–	C
[54]	B-Rep	B-Rep	B-Rep	B-Rep $\rightarrow$ B-Rep	A2	–	–
[11]	Mesh	Implicit (SDF)	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Implicit (SDF) $\rightarrow$ Mesh	A3	11/2	C
[118]	B-Rep	Implicit (Neural SDF)	Mesh	B-Rep $\rightarrow$ Implicit (Neural SDF) $\rightarrow$ Mesh	A3	12	G
[65]	Mesh	Implicit (LDF)	Mesh	Mesh $\rightarrow$ Implicit (LDF) $\rightarrow$ Mesh	A3	12	C
[114]	B-Rep	Implicit (NH-Rep)	Mesh	B-Rep $\rightarrow$ Implicit (NH-Rep) $\rightarrow$ Mesh	A3	13	G
[81]	Mesh	Implicit (Octree UDF)	Mesh	Mesh $\rightarrow$ Implicit (Octree UDF) $\rightarrow$ Mesh	A3	–	C
[40]	Mesh	Implicit (SDF)	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Implicit (UDF) $\rightarrow$ Implicit (SDF) $\rightarrow$ Mesh	A3	–	C
[39]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A3	–	C
[73]	B-Rep / Mesh	Ray-Rep (LDI)	Mesh	B-Rep / Mesh $\rightarrow$ Ray-Rep (LDI) $\rightarrow$ Mesh	A4	–	G
[97]	Mesh	Implicit (RBF+SDF)	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Implicit (RBF+SDF) $\rightarrow$ Mesh	A4	–	C
[6]	Mesh	Implicit (SDF)	Mesh	Mesh $\rightarrow$ Voxel $\rightarrow$ Implicit (SDF) $\rightarrow$ Mesh	A4	–	C
[96]	Point Cloud	Implicit (RBF)	Mesh	Point Cloud $\rightarrow$ Implicit (RBF) $\rightarrow$ Mesh	A4	–	H
[75]	Mesh	HSP	Mesh	Mesh $\rightarrow$ HSP $\rightarrow$ Mesh	A5	11	C
[84]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A5	11	C
[90]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A5	11	C
[72]	Mesh	Ray-Rep (LDNI)	Mesh	Mesh $\rightarrow$ Ray-Rep (LDNI) $\rightarrow$ Mesh	A5	11	C
[37]	Parametric Surface	Parametric Surface	Parametric Surface	Parametric Surface $\rightarrow$ Parametric Surface	A5	11	C
[57]	Subdivision Surface	Subdivision Surface	Subdivision Surface	Subdivision Surface $\rightarrow$ Subdivision Surface	A5	11	C
[106]	Point Cloud	Implicit (Neural SDF)	Mesh	Point Cloud $\rightarrow$ Implicit (Neural SDF) $\rightarrow$ Mesh	A5	13	G
[112]	B-Rep / Point Cloud	Point Cloud	B-Rep / Point Cloud	B-Rep / Point Cloud $\rightarrow$ B-Rep / Point Cloud	A5	13	G
[113]	Implicit (SDF)	Implicit (Neural SDF)	Mesh	Mesh $\rightarrow$ Implicit (Neural SDF) $\rightarrow$ Mesh	A5	–	G
[100]	Mesh / Point Cloud	Implicit (Neural SDF)	Mesh	Mesh / Point Cloud $\rightarrow$ Implicit (Neural SDF) $\rightarrow$ Mesh	A5	–	G
[105]	Point Cloud	Implicit (Neural SDF)	Mesh	Point Cloud $\rightarrow$ Implicit (Neural SDF) $\rightarrow$ Mesh	A5	–	G
[68]	B-Rep / Mesh / CSG / Voxel / Implicit	Ray-Rep	Ray-Rep	B-Rep / Mesh / CSG / Voxel / Implicit $\rightarrow$ Ray-Rep	A5	–	G
[77]	Mesh	Voxel	Voxel	Mesh $\rightarrow$ Voxel	A5	–	G
[58]	B-Rep	B-Rep	B-Rep	B-Rep $\rightarrow$ B-Rep	A5	–	C
[10]	B-Rep	B-Rep	B-Rep	B-Rep $\rightarrow$ B-Rep	A5	–	C
[35]	B-Rep	B-Rep	B-Rep	B-Rep $\rightarrow$ B-Rep	A5	–	C
[41]	Mesh / Point Cloud / Voxel	Implicit (GSD)	Mesh	Mesh / Point Cloud / Voxel $\rightarrow$ Implicit (GSD) $\rightarrow$ Mesh	A5	–	C
[8]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A5	–	C
[53]	Mesh	Mesh	Mesh	Mesh $\rightarrow$ Mesh	A5	–	C
[43]	Parametric Surface	Parametric Surface	Parametric Surface	Parametric Surface $\rightarrow$ Parametric Surface	A5	–	C
[56]	Parametric Surface	Parametric Surface	Parametric Surface	Parametric Surface $\rightarrow$ Parametric Surface	A5	–	C
[55]	Parametric Surface	Parametric Surface	Parametric Surface	Parametric Surface $\rightarrow$ Parametric Surface	A5	–	C
[48]	Parametric Surface	Parametric Surface	Parametric Surface	Parametric Surface $\rightarrow$ Parametric Surface	A5	–	C
[36]	Parametric Surface	Parametric Surface	Parametric Surface	Parametric Surface $\rightarrow$ Parametric Surface	A5	–	C
[66]	Ray-Rep	Ray-Rep	Ray-Rep	Ray-Rep $\rightarrow$ Ray-Rep	A5	–	C
[169]	CSG	Voxel	Mesh	CSG $\rightarrow$ Voxel $\rightarrow$ Mesh	A5	–	C

Note. Acceleration levels A1–A5 indicate the use of computational acceleration structures, from A1 (hybrid acceleration) through A2 (BVH/KD-tree/AABB-tree), A3 (octree/adaptive grid), A4 (uniform or hash grid), to A5 (no acceleration). Iteration levels 11, 12, 13 correspond respectively to < 50, 50–300, and > 300 iterations. Device categories C, G, and H denote CPU, GPU, and Hybrid architectures, respectively. The symbol -- denotes not applicable or not reported. Detailed level definitions are provided in Supplementary Material.

Adaptive Grids [118] or Hash Grids [73]. Neural implicit approaches often bypass explicit acceleration structures, instead relying on neural compression techniques [169] or leveraging the massive throughput of tensor cores to evaluate global functions directly [100], [105], [106], [113], thereby avoiding the management overhead associated with complex spatial hierarchies.

B. Quality

The quality of an offset surface reflects a balance between geometric precision and topological correctness, as summarized in Table VII. Geometric precision describes how accurately the surface conforms to the prescribed offset distance. Specific distance metrics utilizing directional distance fields [170] or implicit

filtering frameworks [115] assist in accurate shape comparisons during optimization and construction stages. During the evaluation phase, we employ general metrics including the Hausdorff distance (d_H), Chamfer distance (d_C), and average distance to quantify deviations from the target geometry. We further distinguish the normal computation methods and the strategy for closest-point queries. Topological validity concerns the generation of a valid, watertight, and non-self-intersecting manifold.

Normal and query: The accuracy of normal estimation and the efficacy of closest-point query strategies influence the final geometric precision of offset surfaces. Exact analytical approaches [10], [43], [58] derive normals directly from parametric definitions or bijective mappings to ensure mathematical exactness. Discrete methods including weighted face normals [85]

TABLE VII
COMPARATIVE ANALYSIS OF GEOMETRIC ACCURACY AND TOPOLOGICAL
VALIDITY INDICATORS

Ref.	Geometric Accuracy			Topological Validity		
	Normal	Query	Accuracy	Self-Int Avoid	Manifold	Watertight
[62]	N0	Q1	—	Y	Y	Y
[75]	N0	Q1	A1/D1	Y	—	—
[66]	N0	—	—	—	—	—
[68]	N0	—	—	Y	Y	Y
[82]	N0	—	—	Y	Y	Y
[64]	N0	—	—	Y	Y	Y
[84]	N0	—	A3	Y	Y	Y
[58]	N1	Q1	—	—	—	—
[35]	N1	Q1	—	P	P	P
[55]	N1	Q1	—	Y	Y	N
[10]	N1	Q1	—	Y	Y	Y
[89]	N1	Q1	—	Y	Y	Y
[90]	N1	Q1	—	Y	Y	Y
[54]	N1	Q4	—	Y	—	—
[112]	N1	Q5	A3 ^d /D1-2	N	N	N
[48]	N1	—	—	—	—	—
[36]	N1	—	—	—	—	—
[56]	N1	—	—	N	N	N
[37]	N1	—	—	N	Y	Y
[8]	N1	—	—	P	P	P
[57]	N1	—	—	Y	—	—
[43]	N1	—	A1 ⁿ /D3	—	—	—
[106]	N2	Q1	—	—	—	Y
[155]	N2	Q1	—	Y	Y	Y
[118]	N2	Q1	A1-2/D1-2	—	Y	—
[65]	N2	Q1	A1-3	Y	Y	Y
[169]	N2	Q2	—	—	—	—
[113]	N2	Q2	—	—	—	—
[111]	N2	Q2	—	Y	Y	Y
[100]	N2	Q2	D1-2	—	—	Y
[105]	N2	—	—	—	—	Y
[41]	N2	—	—	—	Y	Y
[114]	N2	—	A3/D2	—	Y	Y
[81]	N3	Q1	—	N	Y	Y
[97]	N3	Q1	—	Y	—	Y
[80]	N3	Q1	A2	Y	Y	Y
[77]	N3	—	—	—	—	Y
[6]	N3	—	—	P	P	P
[72]	N3	—	A1 ^r	Y	Y	Y
[39]	N3	—	A1-2	Y	Y	Y
[79]	N3/4	Q1	—	Y	—	Y
[40]	N3/4	Q1	D1	Y	Y	Y
[85]	N4	Q1/3/4	A1/D2	Y	Y	Y
[53]	N4	—	—	—	—	—
[96]	N5	—	A3	—	Y	Y
[73]	N5	—	A3/D2	Y	—	—

Note. Geometric accuracy classes A1–A3 indicate high, medium, and low accuracy based on reported Hausdorff, Chamfer, and average distances. Unless otherwise noted, these metrics were normalized against the bounding box diagonal of the input model in the original papers. Exceptions with different normalization scales are annotated: ^r Normalized by the offset distance; ^d Normalized using the minimum bounding box size along the offset direction; ⁿ Dimensionless relative L_2 error based on the coordinate norm of the ground-truth surface. Discretization levels D1–D3 represent high, medium, and low spatial resolution, where S denotes sampling density and R denotes spatial resolution. Specifically, D1 denotes $S > 200k$ or $R \leq 1/512$, D2 denotes $10k < S \leq 200k$ or $1/128 \leq R < 1/512$, and D3 denotes $S < 10k$ or $R > 1/128$. Normal levels N0–N5 specify normal-computation methods: N0 no normals; N1 analytical; N2 SDF-gradient; N3 face normals; N4 weighted face normals; N5 PCA normals. Query levels Q1–Q5 specify closest-point methods: Q1 exact point–triangle; Q2 analytical approximation; Q3 MLS-based; Q4 Newton iterative projection; Q5 KNN approximation. Topological symbols: Y valid; N invalid; P partially valid; The symbol— not applicable or reported. Detailed level definitions are provided in the Supplementary Material.

TABLE VIII
COMPARATIVE ANALYSIS OF ROBUSTNESS UNDER DEFECTIVE INPUTS,
EXTREME PARAMETERS, AND POST-PROCESSING DEPENDENCIES

Ref.	Defective Inputs			Extreme Parameter			PostProc
	NonMF	NonWT	Degen	$\delta \rightarrow 0$	Flip-N	$\delta < 0$	Remesh
[84]	Y	Y	Y	Y	Y	Y	Y
[155]	Y	Y	Y	Y	Y	Y	Y
[41]	Y	Y	Y	Y	Y	Y	O
[106]	Y	Y	Y	Y	Y	Y	N
[105]	Y	Y	Y	Y	Y	Y	N
[96]	Y	Y	Y	Y	Y	Y	—
[80]	Y	Y	Y	Y	Y	N	Y
[40]	Y	Y	Y	Y	Y	—	N
[100]	Y	Y	Y	Y	—	Y	N
[39]	Y	Y	Y	Y	—	Y	N
[64]	Y	Y	Y	—	Y	Y	O
[62]	Y	Y	Y	—	Y	N	N
[75]	Y	Y	Y	—	—	Y	N
[81]	Y	Y	N	Y	Y	Y	Y
[113]	Y	Y	—	Y	N	Y	—
[65]	Y	Y	—	N	Y	Y	O
[118]	N	Y	Y	Y	Y	Y	N
[37]	N	Y	N	Y	—	Y	N
[114]	N	Y	—	Y	—	Y	N
[8]	N	N	Y	N	—	N	O
[6]	N	N	N	Y	Y	Y	Y
[72]	N	N	N	Y	Y	Y	Y
[48]	N	N	N	Y	—	Y	N
[89]	N	N	N	Y	—	Y	Y
[73]	N	N	N	Y	N	Y	Y
[10]	N	N	N	Y	N	Y	N
[35]	N	N	N	Y	N	N	N
[112]	N	N	N	Y	—	Y	N
[43]	N	N	N	Y	—	Y	N
[90]	N	N	N	N	Y	—	N
[56]	N	N	N	—	N	N	N
[66]	N	N	N	—	—	Y	N
[82]	N	N	—	Y	Y	Y	Y
[53]	N	N	—	Y	N	Y	N
[79]	N	N	—	N	N	Y	N
[77]	N	N	—	—	Y	Y	N
[11]	N	N	—	—	N	—	—
[55]	N	—	N	—	N	Y	N
[58]	N	—	—	Y	—	Y	N
[54]	N	—	—	Y	—	Y	N
[68]	—	—	N	N	Y	Y	N
[57]	—	—	N	—	N	Y	N
[85]	—	—	—	Y	—	Y	N
[97]	—	—	—	Y	—	Y	—
[36]	—	—	—	—	—	Y	N
[169]	—	—	—	—	—	Y	—

Note. NonMF: Non-Manifold. NonWT: Non-Watertight. Degen: Degenerate inputs. Flip-N: Normal Flip. $\delta \rightarrow 0$: Infinitesimal offset. $\delta < 0$: Negative offset. For the Defective Inputs and Extreme Parameter columns: Y valid; N invalid. For the Post-Process column: Y required; N not required; O optional. The symbol — denotes not applicable or not reported.

[112], [113], [114]. Furthermore, some research has advanced toward effective function representations without reliance on explicit normal definitions [62], [65], [84].

Topological validity: Early Constructive Methods, particularly those for parametric surfaces, prioritize geometric precision [35], [56]. In theory, they can offer analytical exactness. However, this fidelity comes at a high cost to topological robustness, as these methods are susceptible to generating self-intersections, especially in concave regions where local curvature is high [55]. Furthermore, the offset of a parametric surface is often not representable in the same parametric form. Consequently, these approaches rely on complex, trimming and self-intersection removal algorithms, failing to guarantee topologically valid outputs [54]. By converting the problem into a discrete domain and extracting isosurfaces from scalar fields, spatial discretization and field-based methods implicitly

and PCA-based estimation [96] are constrained by input sampling quality and often exhibit directional ambiguity near sharp features. In field-based methods, normals are typically computed as signed distance field gradients [155]. The resolution constraints of finite difference methods often motivate compensatory mechanisms such as robust point-triangle queries [40], [80] to mitigate discretization errors. Neural implicit architectures utilize automatic differentiation to enforce smoothness priors that enhance robustness against noise [100], [105], [106],

resolve self-intersections. SDF-based [40] and UDF-based approaches [80] produce watertight and manifold outputs, even from defective inputs. However, fidelity is limited by the grid resolution [64], [72], [81]. While some methods provide error bounds [11], their geometric accuracy is typically lower than analytical counterparts.

Sharp feature preservation: Adaptive volumetric methods mitigate the scalability issues of uniform grids by refining spatial structures near geometric discontinuities. Approaches such as FPO [81] and high-resolution techniques [155] utilize topology-adapted octrees to concentrate computational resources. However, their topological validity remains bounded by the minimum voxel size. Structural features thinner than the finest leaf node, such as thin shells or narrow passages are merged or lost due to the topological resolution limit of the underlying grid.

Optimization-based methods in [65], [82], [85] retain high-frequency details by formulating feature reconstruction through guided energy optimization [85], quadratic programming with geometric constraints [82], or by exploiting the inherent smoothness of localized, primitive-based distance fields [65]. These strategies effectively ensure that the resulting offset meshes preserve geometrically sharp corners and edges. Nevertheless, whether relying on iterative evaluations within particle systems [85], dynamic programming and facet intersection calculations [82], or linear field approximations and half-plane cutting within dense tetrahedra [65], these operations are computationally demanding.

Within learning-based methods [100], [105], [106], [113], [114], the loss of sharp features is attributed to the spectral bias of neural networks. Because high-frequency details are associated with exponentially smaller eigenvalues, their convergence rate is slower [104], [171], [172]. As a result, coordinate-based MLPs, particularly those using standard ReLU activation preferentially learn low-frequency smooth components, leading to the loss of high-frequency details [106], [173].

C. Robustness

Robustness considerations permeate the entire surface offsetting pipeline, fundamentally concerning the preservation of geometric validity when confronted with intrinsic defects in input geometry and potential degeneracies introduced by offset parameters. Table VIII documents the performance of offset methods under these challenges, revealing two complementary robustness mechanisms alongside their coupling with post-processing requirements.

Input-driven approaches: The first mechanism prioritizes tolerance to pathological input geometry through implicit regularization via representation conversion. Foundational techniques, such as GWN [120], [133], extend classical mathematical principles to enable robust inside-outside quantification on defective inputs like triangle soups. Building on such theoretical underpinnings, one primary category includes analytic field-based methods [40], [41]. Additionally, this implicit strategy is prevalent in learning-based [100], [106] and spatial discretization-based approaches [80]. These methods systematically avoid direct

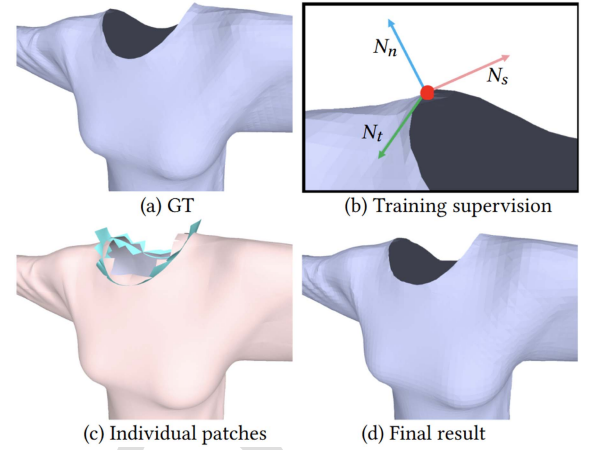


Fig. 14. To reconstruct open boundary surfaces suitable for implicit offsetting, a trimming patch is learned to explicitly define the surface termination. The normal supervision N_s for this patch is derived via the cross product of the surface normal N_n and the boundary tangent N_t . Figure from [118].

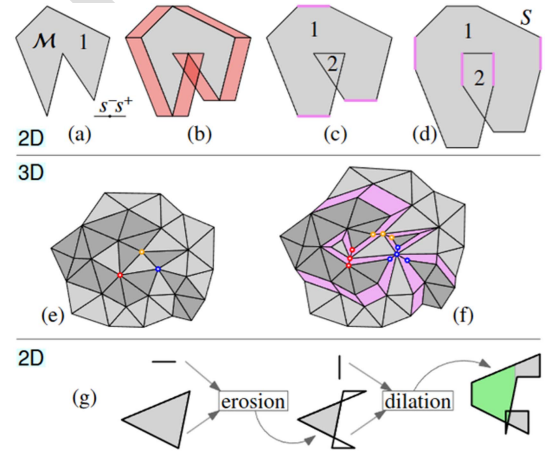


Fig. 15. Continuous offsetting. (a–d): A 2D schematic visualizing two successive dilations: expansion is modeled by attaching prisms to edges (b), followed by topological simplification based on dixel counts (c). (e–f): 3D combinatorial structure: light and dark triangles distinguish front and back facets, which are separated by purple quads derived from extruded silhouette edges. Colored vertices indicate singularities requiring duplication to maintain manifold validity. (g) A counter-example illustrating the irreversibility of continuous offsetting: the algorithmic sequence yields an erroneous shape, failing to produce the correct result (shaded in green). Figure from [68].

manipulation of potentially degenerate explicit geometry. Instead, they project input surfaces onto intermediate representations. This conversion process implicitly resolves topological inconsistencies and intersection artifacts. Consequently, these methods demonstrate high tolerance to non-manifold surfaces, boundary defects, and self-intersections.

Parameter-driven approaches: The second paradigm focuses on the precise management of topological events during the explicit offsetting process. This approach dominates constructive methods and optimization-based techniques [48], [54], [84], [89]. These algorithms operate under strict geometric assumptions, requiring manifold, orientable input surfaces with well-defined normal fields. The algorithmic

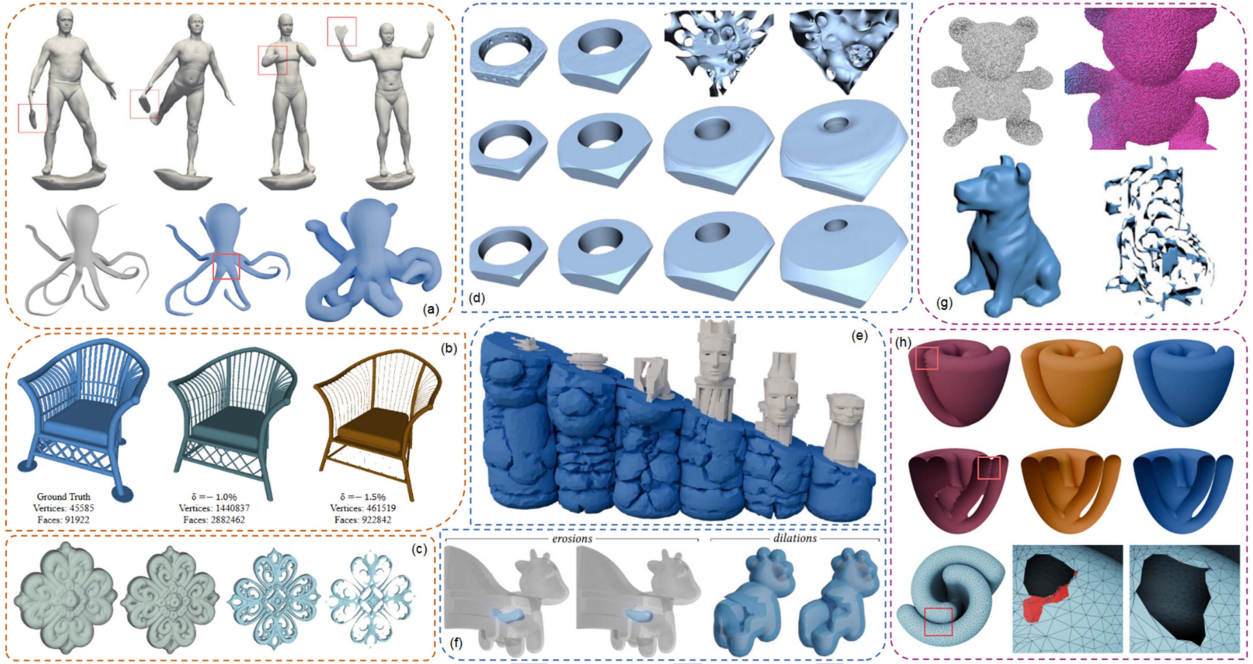


Fig. 16. Common failure cases and challenges in surface offsetting, illustrated with results adapted from the cited literature. (a) Topological merging. *Top*: Erroneous fusion of fingers on the D-Faust dataset [106]. *Bottom*: Fusion of tentacles on an octopus model, shown at zero offset and with a positive offset of $d = 0.02$ [100]. (b) Geometric degradation under negative offsetting. Topological disconnection when negative offset exceeds local wall thickness on a thin-walled chair model. From left to right: $d = 0$ (original), $d = -0.01$, $d = -0.015$ [81]. (c) Feature thinning progression across varying offset distances. From left to right: positive offset $d = 0.05$, $d = 0.025$, original geometry $d = 0$, negative offset $d = -0.0125$ [73]. (d) Artifacts and feature loss in learning-based offsetting. From left to right: $d = -0.1$, $d = 0$ (original), $d = 0.1$, $d = 0.2$. *Top*: The geometry degenerates into topological noise and disconnected artifacts at positive offset $d = 0.1$ and $d = 0.2$ [105]. *Middle*: Sharp features are over-smoothed and surface irregularities emerge at positive offset $d = 0.2$ and 0.2 [106]. *Bottom*: The offset results generated by the method of Guo et al. [114]. (e) Algorithmic non-convergence: Failure to converge on a complex model after 17 hours leads to an incompletely fused and invalid geometry [84]. (f) Incorrect field representation: The computation fails to produce the correct geometric behavior for erosion and dilation operations [113]. (g) Surface fracture and fragmentation. *Top*: Numerical instability and fractures resulting from a 0.2% positive offset on a rough surface [65]. *Bottom*: Coarse 32^3 voxelization leads to a discontinuous and fragmented surface [79], [85]. (h) *Top*: An initial self-intersecting output (left) contrasted with two valid manifold results (middle, right) [84]. *Middle*: The cross-sectional view corresponding to the top row, showing the effect of collision resolution [84]. *Bottom*: Applying a 10% offset to a Thing10 k mesh creates extensive internal self-intersections, where back-faces are rendered red [81].

complexity in this mechanism is concentrated on precise control of geometric degeneracies induced by extreme parameters. Critical to this mechanism is the remesh integration of geometric repair operations [84], [89] as algorithmic components rather than optional post-processing steps.

We have synthesized general patterns linking algorithmic mechanisms to their robustness capabilities. These summaries detail how specific approaches support resilience against input defects and parameter extremes, as presented in the Supplementary Material.

VI. OPEN PROBLEMS

The comparative analysis in Section IV reveals performance balance across method classes. This section investigates representative failure cases that persist across different methods.

Open surfaces: Conventional neural implicit methods primarily rely on zero-level sets to define closed geometries. When processing open surfaces with explicit boundaries, such as the neckline of a garment shown in Fig. 14(a), the boundary lacks a strict sign indicator. While some recent approaches attempt to handle open boundaries by learning UDFs [107] or predicting additional auxiliary masks for boundary extraction [174], they

often struggle to reconstruct clean boundaries. Trimming-based methods [118] address this limitation by introducing a dedicated normal supervision vector (N_s) derived from the cross product of the surface normal (N_n) and the boundary tangent (N_t) to explicitly model the cutting surface (Fig. 14(b)). However, this approach relies on explicitly defining the necessary gradient directions for the implicit function, indicating a dependency on pre-processing rather than learning these features.

Feature eroding: Negative offsetting, or erosion, presents the annihilation of features. As demonstrated in Fig. 16(b)–(c), when the negative offset distance approaches or exceeds the local thickness of a structure, thin-walled components [81] and intricate filigree details [73] are progressively eroded and ultimately eliminated. This issue is fundamentally compounded in continuous offsetting operation, such as a sequence of negative offsetting followed by positive offsetting. As shown in Fig. 15, this process is not geometrically involutive. Martínez et al. [68] demonstrate that while segment-based dilation can be modeled, the composition of erosion and dilation often yields an incorrect result. The green shaded area highlights the topological information that is permanently lost during the negative phase and cannot be recovered by the subsequent positive offset. This irreversibility poses a challenge for applications requiring reversible geometric evolution.

Topological integrity: The loss of topological integrity frequently occurs as the erroneous fusion of distinct geometric features. As illustrated in Fig. 16(a), implicit methods may fail to resolve small gaps, leading to the merging of limbs on the D-Faust dataset [106] or the tentacles of an octopus model, even at a zero offset distance [100]. This issue is not unique to implicit fields; Fig. 16(e) demonstrates an algorithmic failure where an optimization-based method fails to converge, resulting in the incomplete fusion of distinct components [84].

Manifold and artifacts: The loss of manifold validity represents a category of failure, which can present as surface fracture or self-intersection. Fracturing, shown in Fig. 16(g), often results from underlying discretization such as coarse voxelization [79], [85], or from numerical instabilities when an algorithm is applied to non-smooth input geometry [65]. More common is the generation of self-intersections, a canonical problem for positive offsets in concave regions. Fig. 16(h) illustrates this phenomenon, showing both the self-intersecting output of an algorithm [84] and the internal non-manifold geometry generated by offsetting. Additionally, failures related to artifacts and loss of geometric fidelity form another challenge. As visualized in Fig. 16(d), methods utilizing periodic activation functions can degenerate into topological noise at positive offset isovalues [105], while implicit geometric regularization often leads to over-smoothing and surface irregularities [106]. Furthermore, geometric fidelity is compromised by theoretically inexact field operations. As illustrated in Fig. 16(f), the use of standard min/max operators for CSG yields pseudo-SDFs, which locally satisfy the Eikonal equation but fail to represent true global distances. Thus producing mathematically incorrect dilation and erosion behaviors [113].

VII. CONCLUSION

Our comparative analysis across critical dimensions reveals fundamental trade-offs that constrain surface offset algorithmic design. No single method optimally balances geometric exactness with computational robustness. Parametric methods achieve precise distance control but generate invalid topology in concave regions. Volumetric methods guarantee valid manifolds but sacrifice sharp features to discretization. This dichotomy persists because offset operations fundamentally couple local geometric decisions with global topological constraints.

We identify critical unsolved challenges. Open boundary offsetting lacks universally robust solutions. Current approaches typically require explicit geometric interventions, such as predicting auxiliary masks [174] or learning dedicated trimming patches [118] to cut the extended zero-level set, or they face challenges with free-edge artifacts when extracting meshes directly from UDF [107]. Self-intersection handling in concave regions remains fundamentally ill-posed, where existing approaches rely on costly geometric repair or accept controlled manifold violations [54], [84]. Feature preservation under negative offsets fails when offset distance approaches local thickness, causing thin-wall collapse in discrete methods and numerical instability in continuous formulations [62], [65], [73], [81].

Future research will benefit from extending classical offset theory toward more general surface domains, including non-manifold and open-boundary cases, while retaining consistent interior–exterior classification. Explorations could investigate unified field formulations that natively encode boundary singularities. Scalable and numerically stable geometric predicates remain essential for robust self-intersection resolution, where learned heuristics may further enable adaptive refinement at scale. Efficient tracking of topological events during optimization similarly requires data structures supporting dynamic connectivity updates with provable correctness guarantees. In learning-based offsetting, feature preservation motivates architectural designs that disentangle distance–field smoothness from geometric sharpness, such as accurate representations of local geometry [175]. Maintaining consistency during the dynamic geometries constitutes an additional research direction, for example through neural deformation frameworks [176], or by integrating boundary-aware topological loss functions to naturally terminate the offset generation without requiring auxiliary structures.

REFERENCES

- [1] T. Van Hook, “Real-time shaded NC milling display,” *ACM SIGGRAPH Comput. Graph.*, vol. 20, no. 4, pp. 15–20, 1986.
- [2] T. Saito and T. Takahashi, “NC machining with G-buffer method,” *ACM SIGGRAPH Comput. Graph.*, vol. 25, no. 4, pp. 207–216, 1991.
- [3] K. C. Hui, “Solid sweeping in image space—Application in NC simulation,” *Vis. Comput.*, vol. 10, no. 6, pp. 306–316, 1994.
- [4] Y. Xie and X. Chen, “Support-free interior carving for 3D printing,” *Vis. Informat.*, vol. 1, no. 1, pp. 9–15, 2017.
- [5] S. Hornus, S. Lefebvre, J. Dumas, and F. Claux, “Tight printable enclosures and support structures for additive manufacturing,” in *Proc. Eurographics Workshop Graph. Digit. Fabr.*, 2016, pp. 11–21.
- [6] L. Chen, M. F. Chung, Y. Tian, A. Joneja, and K. Tang, “Variable-depth curved layer fused deposition modeling of thin-shells,” *Robot. Comput.-Integr. Manuf.*, vol. 57, pp. 422–434, 2019.
- [7] M. Livesu, S. Ellero, J. Martínez, S. Lefebvre, and M. Attene, “From 3D models to 3D prints: An overview of the processing pipeline,” *Comput. Graph. Forum*, vol. 36, no. 2, pp. 537–564, 2017.
- [8] R. Aubry, S. Dey, E. Mestreau, and B. Karamete, “Boundary layer mesh generation on arbitrary geometries,” *Int. J. Numer. Methods Eng.*, vol. 112, no. 2, pp. 157–173, 2017.
- [9] C. Zhang, M. Elgharib, G. Fox, M. Gu, C. Theobalt, and W. Wang, “An implicit parametric morphable dental model,” *ACM Trans. Graph.*, vol. 41, no. 6, pp. 1–13, 2022.
- [10] J. R. Rossignac and A. A. Requicha, “Offsetting operations in solid modelling,” *Comput. Aided Geometric Des.*, vol. 3, no. 2, pp. 129–148, 1986.
- [11] G. Varadhan and D. Manocha, “Accurate minkowski sum approximation of polyhedral models,” in *Proc. 12th Pacific Conf. Comput. Graph. Appl.*, 2004, pp. 392–401.
- [12] J.-M. Lien, “Covering minkowski sum boundary using points with applications,” *Comput. Aided Geometric Des.*, vol. 25, no. 8, pp. 652–666, 2008.
- [13] S. Sellán, J. Kesten, A. Y. Sheng, and A. Jacobson, “Opening and closing surfaces,” *ACM Trans. Graph.*, vol. 39, pp. 1–13, 2020.
- [14] B. Pham, “Offset curves and surfaces: A brief survey,” *Comput.-Aided Des.*, vol. 24, pp. 223–229, 1992.
- [15] T. Maekawa, “An overview of offset curves and surfaces,” *Comput.-Aided Des.*, vol. 31, no. 3, pp. 165–173, 1999.
- [16] M. A. Kulczycka and L. J. Nachman, “Qualitative and quantitative comparisons of B-spline offset surface approximation methods,” *Comput.-Aided Des.*, vol. 34, no. 1, pp. 19–26, 2002.
- [17] B. Delaunay, “Sur la sphère vide. a la mémoire de georges voronoï,” *Proc. Russian Acad. Sci.*, vol. 6, pp. 793–800, 1934.

- [18] P. Lancaster and K. Salkauskas, "Surfaces generated by moving least squares methods," *Math. Comput.*, vol. 37, no. 155, pp.141–158, 1981.
- [19] M. Kazhdan, M. Bolitho, and H. Hoppe, "Poisson surface reconstruction," in *Proc. 4th Eurographics Symp. Geometry Process.*, 2006, pp. 61–70.
- [20] L. We, "Marching cubes: A high resolution 3D surface construction algorithm," *Comput. Graph.*, vol. 21, no. 1, pp. 7–12, 1987.
- [21] T. Ju, F. Losasso, S. Schaefer, and J. Warren, "Dual contouring of hermite data," in *Proc. 29th Annu. Conf. Comput. Graph. Interactive Techn.*, 2002, pp. 339–346.
- [22] M. Berger et al., "A survey of surface reconstruction from point clouds," in *Comput. Graph. Forum*, vol. 36, no. 1, pp. 301–329, 2017.
- [23] H. Hoppe, T. DeRose, T. Duchamp, J. McDonald, and W. Stuetzle, "Surface reconstruction from unorganized points," in *Proc. 19th Annu. Conf. Comput. Graph. Interactive Techn.*, 1992, pp. 71–78.
- [24] M. Botsch and L. Kobbelt, "A remeshing approach to multiresolution modeling," in *Proc. Eurographics/ACM SIGGRAPH Symp. Geometry Process.*, 2004, pp. 185–192.
- [25] J. Rossignac and A. A. Requicha, "Solid modeling," 1999.
- [26] A. Kaufman, D. Cohen, and R. Yagel, "Volume graphics," *Computer*, vol. 26, no. 7, pp. 51–64, 1993.
- [27] L. P. Kobbelt, M. Botsch, U. Schwaner, and H.-P. Seidel, "Feature sensitive surface extraction from volume data," in *Proc. 28th Annu. Conf. Comput. Graph. Interactive Techn.*, 2001, pp. 57–66.
- [28] A. G. Requicha, "Representations for rigid solids: Theory, methods, and systems," *ACM Comput. Surv.*, vol. 12, no. 4, pp. 437–464, 1980.
- [29] A. Ricci, "A constructive geometry for computer graphics," *Comput. J.*, vol. 16, no. 2, pp. 157–160, 1973.
- [30] V. Shapiro and D. L. Vossler, "Efficient CSG representations of two-dimensional solids," 1991.
- [31] J. Bloomenthal and C. Bajaj, *Introduction to Implicit Surfaces*. San Mateo, CA, USA: Morgan Kaufmann, 1997.
- [32] B. R. de Araújo, D. S. Lopes, P. Jepp, J. A. Jorge, and B. Wyvill, "A survey on implicit surface polygonization," *ACM Comput. Surv.*, vol. 47, no. 4, pp. 1–39, 2015.
- [33] E. L. Brechner, "General tool offset curves and surfaces," in *Proc. ASME Des. Tech. Conf.*, 1990.
- [34] H. Pottmann, "General offset surfaces," *Neural Parallel Sci. Comput.*, vol. 5, no. 1/2, pp. 55–80, 1997.
- [35] R. T. Farouki, "Exact offset procedures for simple solids," *Comput. Aided. Geom. Des.*, vol. 2, pp. 257–279, 1985.
- [36] Y. Yue and J. Jia, "Computing offsets of nurbs curve and surface," *Adv. Mater. Res.*, vol. 542/543, pp. 537–540, 2012.
- [37] B. Bastl, B. Jüttler, J. Kosinka, and M. Lávička, "Computing exact rational offsets of quadratic triangular bézier surface patches," *Comput.-Aided Des.*, vol. 40, no. 2, pp. 197–209, 2008.
- [38] P. Hachenberger, "Exact minkowski sums of polyhedra and exact and efficient decomposition of polyhedra in convex pieces," in *Proc. Eur. Symp. Algorithms*, 2007, pp. 669–680.
- [39] M. Campen and L. Kobbelt, "Polygonal boundary evaluation of minkowski sums and swept volumes," *Comput. Graph. Forum*, vol. 29, no. 5, pp. 1613–1622, 2010.
- [40] H. Xu and J. Barbič, "Signed distance fields for polygon soup meshes," in *Graphics Interface*, Boca Raton, FL, USA: AK Peters/CRC Press, 2020, pp. 35–41.
- [41] N. Feng and K. Crane, "A heat method for generalized signed distance," *ACM Trans. Graph.*, vol. 43, no. 4, pp. 1–19, 2024.
- [42] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove, "DeepSDF: Learning continuous signed distance functions for shape representation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 165–174.
- [43] R. Akhrif, A. Kouibia, and M. Pasadas, "Approximation of generalized offset surfaces by bicubic splines," *J. Math. Chem.*, vol. 58, no. 3, pp. 647–662, 2020.
- [44] P. Musialski, T. Auzinger, M. Birsak, M. Wimmer, and L. Kobbelt, "Reduced-order shape optimization using offset surfaces," *ACM Trans. Graph.*, vol. 34, no. 4, pp. 1–9, 2015.
- [45] D. Dragomatz and S. Mann, "A classified bibliography of literature on NCC milling path generation," *Comput.-Aided Des.*, vol. 29, no. 3, pp. 239–247, 1997.
- [46] D. O. Yang and H.-Y. Feng, "Machining triangular mesh surfaces via mesh offset based tool paths," *Comput.-Aided Des. Appl.*, vol. 5, no. 1/4, pp. 254–265, 2008.
- [47] A. Tagliasacchi, T. Delame, M. Spagnuolo, N. Amenta, and A. Telea, "3D skeletons: A state-of-the-art report," *Comput. Graph. Forum*, vol. 35, no. 2, pp. 573–597, 2016.
- [48] R. T. Farouki, "The approximation of non-degenerate offset surfaces," *Comput. Aided Geom. Des.*, vol. 3, pp. 15–43, 1986.
- [49] G. Varadhan, S. Krishnan, T. Sriram, and D. Manocha, "Topology preserving surface extraction using adaptive subdivision," in *Proc. Eurographics/ACM SIGGRAPH Symp. Geometry Process.*, 2004, pp. 235–244.
- [50] P. Hachenberger, "3D minkowski sum of polyhedra," CGAL User and Reference Manual. CGAL Editorial Board, 2024. [Online]. Available: <https://doc.cgal.org/6.0.1/Manual/packages.html#PkgMinkowskiSum3>
- [51] M. Campen and L. Kobbelt, "Exact and robust (self-)intersections for polygonal meshes," *Comput. Graph. Forum*, vol. 29, no. 2, pp. 397–406, 2010.
- [52] B. Wang et al., "Exact and efficient polyhedral envelope containment check," *ACM Trans. Graph.*, vol. 39, no. 4, 2020, Art. no. 114.
- [53] X. Qu and B. Stucker, "A 3D surface offset method for STL-format models," *Rapid Prototyping J.*, vol. 9, no. 3, pp. 133–141, 2003.
- [54] G. V. V. Ravi Kumar, K. G. Shastri, and B. G. Prakash, "Computing constant offsets of a NURBS B-Rep," *Comput.-Aided Des.*, vol. 35, no. 10, pp. 935–944, 2003.
- [55] Y. F. Sun, A. Y. C. Nee, and K. S. Lee, "Modifying free-formed NURBS curves and surfaces for offsetting without local self-intersection," *Comput.-Aided Des.*, vol. 36, no. 12, pp. 1161–1169, Oct. 2004.
- [56] H. P. Moon, "Equivolumetric offset surfaces," *Comput. Aided Geometric Des.*, vol. 26, no. 1, pp. 17–36, 2009.
- [57] J. Wang and F. Cheng, "A heuristic offsetting scheme for catmull-clark subdivision surfaces," *Comput.-Aided Des. Appl.*, vol. 12, no. 6, pp. 765–771, 2015.
- [58] C. Jin, X. Ge, B. Li, J. Wang, Y. Gao, and Y. Xie, "General overall offset method of complex car-body surface model," in *Proc. Int. Conf. Big Data IoT Cloud Comput.*, 2022, pp. 1–6.
- [59] R. A. Abdel-Baky and F. Mofarreh, "A study on the bertrand offsets of timelike ruled surfaces in minkowski 3-space," *Symmetry*, vol. 14, no. 4, 2022, Art. no. 783.
- [60] A. A. Almoneef and R. A. Abdel-Baky, "Bertrand offsets of slant ruled surfaces in euclidean 3-space," *Symmetry*, vol. 16, no. 2, 2024, Art. no. 235.
- [61] G. U. Kaymanli, "Characterization of the evolute offset of ruled surfaces with b-darboux frame," *J. New Theory*, vol. 33, no. 33, pp. 50–55, 2020. [Online]. Available: <https://dergipark.org.tr/en/pub/jnt/issue/53549/716388>
- [62] C. Portaner, M. Rouxel-Labbé, M. Hemmer, D. Cohen-Steiner, and P. Alliez, "Alpha wrapping with an offset," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–22, 2022.
- [63] I. Boada, N. Coll, N. Madern, and J. Antoni Sellares, "Approximations of 2D and 3D generalized Voronoi diagrams," *Int. J. Comput. Math.*, vol. 85, no. 7, pp. 1003–1022, 2008.
- [64] V. K. Suriyababu, C. Vuik, and M. Möller, "Towards a high quality shrink wrap mesh generation algorithm using mathematical morphology," *Comput.-Aided Des.*, vol. 164, 2023, Art. no. 103608.
- [65] L. Wang et al., "PCO: Precision-controllable offset surfaces with sharp features," *ACM Trans. Graph.*, vol. 43, no. 6, pp. 1–16, 2024.
- [66] E. E. Hartquist, J. Menon, K. Suresh, H. B. Voelcker, and J. Zagajac, "A computing strategy for applications involving offsets, sweeps, and minkowski operations," *Comput.-Aided Des.*, vol. 31, no. 3, pp. 175–183, 1999.
- [67] C. C. Wang, "Computing on rays: A parallel approach for surface mesh modeling from multi-material volumetric data," *Comput. Ind.*, vol. 62, no. 7, pp. 660–671, 2011.
- [68] J. Martinez, S. Hornus, F. Claux, and S. Lefebvre, "Chained segment offsetting for ray-based solid representations," *Comput. Graph.*, vol. 46, pp. 36–47, 2015.
- [69] J. Shade, S. Gortler, L.-W. He, and R. Szeliski, "Layered depth images," in *Proc. 25th Annu. Conf. Comput. Graph. Interactive Techn.*, 1998, pp. 231–242.
- [70] C. C. Wang, Y.-S. Leung, and Y. Chen, "Solid modeling of polyhedral objects by layered depth-normal images on the GPU," *Comput.-Aided Des.*, vol. 42, no. 6, pp. 535–544, 2010.
- [71] Y. Chen and C. C. Wang, "Layer depth-normal images for complex geometries: Part one—Accurate modeling and adaptive sampling," in *Proc. Int. Des. Eng. Tech. Conf. Comput. Inf. Eng. Conf.*, 2008, pp. 717–728.
- [72] Y. Chen and C. C. Wang, "Uniform offsetting of polygonal model based on layered depth-normal images," *Comput.-Aided Des.*, vol. 43, no. 1, pp. 31–46, 2011.
- [73] C. C. Wang and D. Manocha, "GPU-based offset surface computation using point samples," *Comput.-Aided Des.*, vol. 45, no. 2, pp. 321–330, 2013.

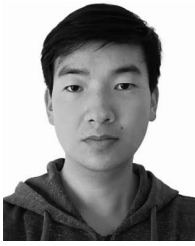
- [74] W. K. Chiu and S. Tan, "Using dexels to make hollow models for rapid prototyping," *Comput.-Aided Des.*, vol. 30, no. 7, pp. 539–547, 1998.
- [75] Z. Chen, D. Panozzo, and J. Dumas, "Half-space power diagrams and discrete surface offsets," *IEEE Trans. Vis. Comput. Graph.*, vol. 26, no. 10, pp. 2970–2981, Oct. 2020.
- [76] D. E. Breen and S. Mauch, "Generating shaded offset surfaces with distance, closest-point and color vol.," in *Proc. Int. Workshop Volume Graph.*, 1999, pp. 307–320.
- [77] W. Li and S. McMains, "Voxelized minkowski sum computation on the GPU with robust culling," *Comput.-Aided Des.*, vol. 43, no. 10, pp. 1270–1283, 2011.
- [78] Y. Chen, H. Wang, D. W. Rosen, and J. Rossignac, "A point-based offsetting method of polygonal meshes," *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 3, pp. 221–235, 2005.
- [79] S. Liu and C. C. Wang, "Fast intersection-free offset surface generation from freeform models with triangular meshes," *IEEE Trans. Automat. Sci. Eng.*, vol. 8, no. 2, pp. 347–360, Apr. 2011.
- [80] Z. Chen, Z. Pan, K. Wu, E. Vouga, and X. Gao, "Robust low-poly meshing for general 3D models," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–20, 2023.
- [81] D. Zint, N. Maruani, M. Rouxel-Labbé, and P. Alliez, "Feature-preserving offset mesh generation from topology-adapted octrees," *Comput. Graph. Forum*, vol. 42, no. 5, 2023, Art. no. e14906.
- [82] H. Cao, G. Xu, R. Gu, J. Xu, X. Zhang, and T. Rabczuk, "A parallel feature-preserving mesh variable offsetting method with dynamic programming," 2023, *arXiv:2310.08997*.
- [83] H. Cao et al., "Robust and feature-preserving offset meshing," 2024, *arXiv:2412.15564*.
- [84] D. Zint, Z. Chen, Y. Zhu, D. Zorin, T. Schneider, and D. Panozzo, "Topological offsets," *ACM Trans. Graph.*, vol. 44, no. 4, pp. 1–19, 2025.
- [85] W. Meng, S. Chen, Z. Shu, S.-Q. Xin, H. Fu, and C. Tu, "Efficiently computing feature-aligned and high-quality polygonal offset surfaces," *Comput. Graph.*, vol. 70, pp. 62–70, 2018.
- [86] A. Kouibia and M. Pasadas, "Reconstruction approximating method by biquadratic splines of offset surfaces holes," *J. Math. Chem.*, vol. 60, no. 2, pp. 423–439, 2022.
- [87] S. D. Porumbescu, B. Budge, L. Feng, and K. I. Joy, "Shell maps," *ACM Trans. Graph.*, vol. 24, no. 3, pp. 626–633, 2005.
- [88] L. Sacht, E. Vouga, and A. Jacobson, "Nested cages," *ACM Trans. Graph.*, vol. 34, no. 6, pp. 1–14, 2015.
- [89] Z. Jiang, T. Schneider, D. Zorin, and D. Panozzo, "Bijective projection in a shell," *ACM Trans. Graph.*, vol. 39, no. 6, pp. 1–18, 2020.
- [90] S. Liu, Y. Ji, J.-P. Guo, L. Liu, and X.-M. Fu, "Smooth bijective projection in a high-order shell," *ACM Trans. Graph.*, vol. 43, no. 4, pp. 1–13, 2024.
- [91] M. W. Jones, J. A. Baerentzen, and M. Sramek, "3D distance fields: A survey of techniques and applications," *IEEE Trans. Vis. Comput. Graph.*, vol. 12, no. 4, pp. 581–599, Jul./Aug. 2006.
- [92] S. F. Frisken, R. N. Perry, A. P. Rockwood, and T. R. Jones, "Adaptively sampled distance fields: A general representation of shape for computer graph.," in *Proc. 27th Annu. Conf. Comput. Graph. Interactive Techn.*, 2000, pp. 249–254.
- [93] Y. Ohtake, A. Belyaev, M. Alexa, G. Turk, and H.-P. Seidel, "Multi-level partition of unity implicits," in *Proc. ACM SIGGRAPH Courses*, 2005, pp. 173–181.
- [94] C. Öztireli, G. Guennebaud, and M. Gross, "Feature preserving point set surfaces based on non-linear kernel regression," in *Comput. Graph. Forum*, vol. 28, no. 2, pp. 493–501, 2009.
- [95] J. C. Carr et al., "Reconstruction and representation of 3D objects with radial basis functions," in *Proc. 28th Annu. Conf. Comput. Graph. Interactive Techn.*, 2001, pp. 67–76.
- [96] S. Liu and C. C. Wang, "Duplex fitting of zero-level and offset surfaces," *Comput.-Aided Des.*, vol. 41, no. 4, pp. 268–281, 2009.
- [97] D. J. Yoo, "General 3D offsetting of a triangular net using an implicit function and the distance fields," *Int. J. Precis. Eng. Manuf.*, vol. 10, pp. 131–142, 2009.
- [98] M. Kazhdan and H. Hoppe, "Screened poisson surface reconstruction," *ACM Trans. Graph.*, vol. 32, no. 3, pp. 1–13, 2013.
- [99] K. Crane, C. Weischedel, and M. Wardetzky, "Geodesics in heat: A new approach to computing distance based on heat flow," *ACM Trans. Graph.*, vol. 32, no. 5, pp. 1–11, 2013.
- [100] G. Coiffier and L. Béthune, "1-lipschitz neural distance fields," *Comput. Graph. Forum*, vol. 43, no. 5, 2024, Art. no. e15128.
- [101] V. Sitzmann, M. Zollhöfer, and G. Wetzstein, "Scene representation networks: Continuous 3D-structure-aware neural scene representations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1119–1130.
- [102] M. Atzmon and Y. Lipman, "SAL: Sign agnostic learning of shapes from raw data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 2562–2571.
- [103] V. Sitzmann, E. Chan, R. Tucker, N. Snavely, and G. Wetzstein, "MetaSDF: Meta-learning signed distance functions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 10136–10147.
- [104] M. Tancik et al., "Fourier features let networks learn high frequency functions in low dimensional domains," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 7537–7547.
- [105] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein, "Implicit neural representations with periodic activation functions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 7462–7473.
- [106] A. Gropp, L. Yariv, N. Haim, M. Atzmon, and Y. Lipman, "Implicit geometric regularization for learning shapes," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3789–3799.
- [107] J. Chibane et al., "Neural unsigned distance fields for implicit function learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21638–21652.
- [108] S. Ren, J. Hou, X. Chen, Y. He, and W. Wang, "GeoUDF: Surface reconstruction from 3D point clouds via geometry-guided distance representation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 14214–14224.
- [109] C. Xu, F. Hou, W. Wang, H. Qin, Z. Zhang, and Y. He, "Details enhancement in unsigned distance field learning for high-fidelity 3D surface reconstruction," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 8806–8814.
- [110] J. Hu et al., "A lightweight UDF learning framework for 3D reconstruction based on local shape functions," 2024, *arXiv:2407.01330*.
- [111] X. Chen, F. Hou, W. Wang, H. Qin, and Y. He, "MIND: Material interface generation from UDFs for non-manifold surface reconstruction," 2025, *arXiv:2506.02938*.
- [112] A. D. Prasad, A. Balu, H. Shah, S. Sarkar, and A. Krishnamurthy, "NURBS-Diff: A differentiable NURBS layer for machine learning cad applications," *Comput.-Aided Des.*, vol. 132, 2021, Art. no. 102970.
- [113] Z. Marschner, S. Sellán, H.-T. D. Liu, and A. Jacobson, "Constructive solid geometry on neural signed distance fields," in *Proc. SIGGRAPH Asia Conf. Papers*, 2023, pp. 1–12.
- [114] H.-X. Guo, Y. Liu, H. Pan, and B. Guo, "Implicit conversion of manifold B-Rep solids by neural halfspace representation," *ACM Trans. Graph.*, vol. 41, no. 6, pp. 1–15, 2022.
- [115] S. Li, G. Gao, Y. Liu, M. Gu, and Y.-S. Liu, "Implicit filtering for learning neural signed distance functions from 3D point clouds," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 234–251.
- [116] K. Genova, F. Cole, A. Sud, A. Sarna, and T. Funkhouser, "Local deep implicit functions for 3D shape," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4856–4865.
- [117] R. Chhabra et al., "Deep local shapes: Learning local SDF priors for detailed 3D reconstruction," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 608–625.
- [118] G. Lin et al., "Patch-grid: An efficient and feature-preserving neural implicit surface representation," *ACM Trans. Graph.*, vol. 44, no. 2, pp. 1–21, 2025.
- [119] CGAL The Project, "CGAL user and reference manual," CGAL Editorial Board, 2020. [Online]. Available: <https://doc.cgal.org/5.0.3/Manual/packages.html>
- [120] A. Jacobson, L. Kavan, and O. Sorkine-Hornung, "Robust inside-outside segmentation using generalized winding numbers," *ACM Trans. Graph.*, vol. 32, no. 4, pp. 1–12, 2013.
- [121] N. Sharp et al., "Geometrycentral: A modern C library of data structures and algorithms for geometry processing," 2019.
- [122] B. Lévy, "Geogram: A programming library with geometric algorithms," version 1.9.7, Symposium on Geometry Processing Software Award 2023, 2019. [Online]. Available: <https://github.com/BrunoLevy/geogram>
- [123] D. Coeurjolly et al., "DGtal: Digital geometry tools and algorithms library," 2016.
- [124] M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, and B. Lévy, *Polygon Mesh Processing*. Boca Raton, FL, USA: CRC Press, 2010.
- [125] S. Hang, "TetGen, a delaunay-based quality tetrahedral mesh generator," *ACM Trans. Math. Softw.*, vol. 41, no. 2, 2015, Art. no. 11.
- [126] Y. Hu, Q. Zhou, X. Gao, A. Jacobson, D. Zorin, and D. Panozzo, "Tetrahedral meshing in the wild," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–13, 2018.
- [127] L. Diazzi, D. Panozzo, A. Vaxman, and M. Attene, "Constrained delaunay tetrahedrization: A robust and practical approach," *ACM Trans. Graph.*, vol. 42, no. 6, pp. 1–15, 2023.

- [128] Y. Liu et al., "On centroidal voronoi tessellation—Energy smoothness and fast computation," *ACM Trans. Graph.*, vol. 28, no. 4, pp. 1–17, 2009.
- [129] S. Van der Walt et al., "scikit-image: Image processing in python," *PeerJ*, vol. 2, 2014, Art. no. e453.
- [130] G. Guennebaud et al., "Eigen v3," 2010.
- [131] J. Reinders, *Intel Threading Building Blocks: Outfitting C for Multi-Core Processor Parallelism*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2007.
- [132] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Math. Program. Comput.*, vol. 12, no. 4, pp. 637–672, 2020.
- [133] G. Barill, N. G. Dickson, R. Schmidt, D. I. W. Levin, and A. Jacobson, "Fast winding numbers for soups and clouds," *ACM Trans. Graph.*, vol. 37, no. 4, pp. 1–12, 2018.
- [134] D. M. Mount, "ANN: A library for approximate nearest neighbor searching," 2010. [Online]. Available: <http://www.cs.umd.edu/mount/ANN/>
- [135] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, "Fast proximity queries with swept sphere volumes," *Tech. Rep. TR99-018*, 1999.
- [136] J. Blanco and P. Rai, "nanoflann: A C header-only fork of FLANN, a library for nearest neighbor (NN) with kd-trees, 2014," 2017. [Online]. Available: <https://github.com/jlblancoc/nanoflann>
- [137] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [138] M. Attene, "Indirect predicates for geometric constructions," *Comput.-Aided Des.*, vol. 126, 2020, Art. no. 102856.
- [139] P. Cignoni, C. Rocchini, and R. Scopigno, "METRO: Measuring error on simplified surfaces," in *Comput. Graph. Forum*, vol. 17, no. 2, pp. 167–174, 1998.
- [140] S. Dey, R. M. Aubry, B. K. Karamete, and E. L. Mestreau, "Capstone: A geometry-centric platform to enable physics-based simulation and system design," *Comput. Sci. Eng.*, vol. 18, no. 1, pp. 32–39, 2016.
- [141] S. Koch et al., "ABC: A big CAD model dataset for geometric deep learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 9593–9603.
- [142] K. D. D. Willis et al., "Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences," *ACM Trans. Graph.*, vol. 40, no. 4, pp. 1–24, 2021.
- [143] J. G. Lambourne, K. D. D. Willis, P. K. Jayaraman, A. Sanghi, P. Meltzer, and H. Shayani, "BRRepNet: A topological message passing system for solid models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 12768–12777.
- [144] Z. Fan, L. Zhu, H. Li, X. Chen, S. Zhu, and P. Tan, "FloorPlanCAD: A large-scale CAD drawing dataset for panoptic symbol spotting," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 10108–10117.
- [145] Z. Wu et al., "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1912–1920.
- [146] A. X. Chang et al., "ShapeNet: An information-rich 3D model repository," 2015, *arXiv:1512.03012*.
- [147] K. Mo et al., "PartNet: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 909–918.
- [148] Q. Zhou and A. Jacobson, "Thing10k: A dataset of 10,000 3D-printing models," 2016, *arXiv:1605.04797*.
- [149] A. Seff, Y. Ovadia, W. Zhou, and R. P. Adams, "SketchGraphs: A large-scale dataset for modeling relational geometry in computer-aided design," 2020, *arXiv:2007.08506*.
- [150] R. Wu, C. Xiao, and C. Zheng, "DeepCAD: A deep generative network for computer-aided design models," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 6752–6762.
- [151] H. Fu et al., "3D-future: 3D furniture shape with texture," *Int. J. Comput. Vis.*, vol. 129, no. 12, pp. 3313–3337, 2021.
- [152] P. Shilane, P. Min, M. Kazhdan, and T. Funkhouser, "The princeton shape benchmark," in *Proc. Shape Model. Appl.*, 2004, pp. 167–178.
- [153] W. Cao, T. Robinson, Y. Hua, F. Boussuge, A. R. Colligan, and W. Pan, "Graph representation of 3D CAD models for machining feature recognition with deep learning," in *Proc. Int. Des. Eng. Tech. Conf. Comput. Inf. Eng. Conf.*, 2020, p. V11AT11A003.
- [154] A. R. Colligan, T. T. Robinson, D. C. Nolan, Y. Hua, and W. Cao, "Hierarchical CADNet: Learning from b-reps for machining feature recognition," *Comput.-Aided Des.*, vol. 147, 2022, Art. no. 103226.
- [155] D. Pavić and L. Kobbelt, "High-resolution volumetric computation of offset surfaces with feature preservation," *Comput. Graph. Forum*, vol. 27, no. 2, pp. 165–174, 2008.
- [156] Y. Liao, S. Donne, and A. Geiger, "Deep marching cubes: Learning explicit surface representations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2916–2925.
- [157] X. Chen, Z. Wang, J. Liu, C. Gong, and Y. Pang, "A neural network-based mesh quality indicator for three-dimensional cylinder modelling," *Entropy*, vol. 24, no. 9, 2022, Art. no. 1245.
- [158] T. Shen, J. Gao, K. Yin, M.-Y. Liu, and S. Fidler, "Deep marching tetrahedra: A hybrid representation for high-resolution 3D shape synthesis," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 6087–6101.
- [159] T. Shen et al., "Flexible isosurface extraction for gradient-based mesh optimization," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–16, 2023.
- [160] D. Ren, H. Shi, J. Zheng, and J. Cai, "McGrids: Monte Carlo-driven adaptive grids for ISO-surface extraction," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 127–144.
- [161] A. Binninger, R. Wiersma, P. Herholz, and O. Sorkine-Hornung, "TetWeave: Isosurface extraction using on-the-fly delaunay tetrahedral grids for gradient-based mesh optimization," *ACM Trans. Graph.*, vol. 44, no. 4, pp. 1–19, 2025.
- [162] T. Maekawa, W. Cho, and N. M. Patrikalakis, "Computation of self-intersections of offsets of Bézier surface patches," *J. Mech. Des.*, vol. 119, no. 2, pp. 275–283, Jun. 1997.
- [163] J. Xu, Y. Sun, and L. Zhang, "A mapping-based approach to eliminating self-intersection of offset paths on mesh surfaces for CNC machining," *Comput.-Aided Des.*, vol. 62, pp. 131–142, May 2015.
- [164] H. Shen, J. Fu, Z. Chen, and Y. Fan, "Generation of offset surface for tool path in NC machining through level set methods," *Int. J. Prod. Res.*, vol. 48, no. 3, pp. 785–799, Feb. 2010.
- [165] W. Jung, H. Shin, and B. K. Choi, "Self-intersection removal in triangular mesh offsetting," *Comput.-Aided Des. Appl.*, vol. 1, no. 1/4, pp. 477–484, 2004.
- [166] S. Schaefer, T. Ju, and J. Warren, "Manifold dual contouring," *IEEE Trans. Vis. Comput. Graph.*, vol. 13, no. 3, pp. 610–619, May/Jun. 2007.
- [167] T. Rashid, S. Sultana, and M. A. Audette, "Watertight and 2-manifold surface meshes using dual contouring with tetrahedral decomposition of grid cubes," *Procedia Eng.*, vol. 163, pp. 136–148, 2016.
- [168] Z. Chen and H. Zhang, "Neural marching cubes," *ACM Trans. Graph.*, vol. 40, no. 6, pp. 1–15, 2021.
- [169] S. Ren, J. Hou, W. Lin, and W. Wang, "Neural compression for 3D geometry sets," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 25294–25304.
- [170] S. Ren, J. Hou, X. Chen, H. Xiong, and W. Wang, "DDM: A metric for comparing 3D shapes using directional distance fields," 2024, *arXiv:2401.09736*.
- [171] N. Rahaman et al., "On the spectral bias of neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 5301–5310.
- [172] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [173] P.-S. Wang, Y. Liu, Y.-Q. Yang, and X. Tong, "Spline positional encoding for learning 3D implicit signed distance fields," 2021, *arXiv:2106.01553*.
- [174] W. Chen, C. Lin, W. Li, and B. Yang, "3PSDF: Three-pole signed distance function for learning surfaces with arbitrary topologies," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 18522–18531.
- [175] S. Ren and J. Hou, "Shape as line segments: Accurate and flexible implicit surface representation," in *Proc. 13th Int. Conf. Learn. Representations*, 2025.
- [176] Y. Yao, S. Ren, J. Hou, Z. Deng, J. Zhang, and W. Wang, "DynoSurf: Neural deformation-based temporally consistent dynamic surface reconstruction," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 271–288.



Xuyi Zhao (Graduate Student Member, IEEE) received the BS degree in computer science and technology from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2017, and the MS degree in software engineering from the University of Sydney, Sydney, NSW, Australia, in 2023. She is currently working toward the PhD degree with the College of Software, Nankai University, Tianjin, China. Her research interests include 3D shape modeling, geometric construction, and neural implicit representations.

1538
1539
1540
1541
1542
1543
1544
1545



Yuanrui Yang received the BS degree in software engineering from Nanjing University, Nanjing, China, in 2020. He is currently working toward the MS degree in software engineering with the School of Software, Tsinghua University, Beijing, China. His research interests include computer-aided design (CAD) and geometric modeling.



Enya Shen received the BS degree from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2008, and the MS and PhD degrees from the National University of Defense Technology, Changsha, China, in 2010 and 2014, respectively. He is currently an associate research fellow with the School of Software, Tsinghua University, Beijing, China. His research interests include geometry modeling, computer graphics, and 3D AIGC.

1558
1559
1560
1561
1562
1563
1564
1565
1566
1567

1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557



Jianmin Wang received the BS degree from Peking University, China, in 1990, and the ME and PhD degrees in computer software from Tsinghua University, China, in 1992 and 1995, respectively. He is currently a Full Professor with the School of Software, Tsinghua University. His research interests include 3D geometric modeling and industrial software systems. In collaboration with the National Engineering Research Center for Big Data Software, he leads the research and development of advanced industrial software ecosystems.